



**ADDIS ABABA SCIENCE AND TECHNOLOGY UNIVERSITY**

**COLLEGE OF ENGINEERING**

**DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING**

Final Year Project on

*Ethio-Virtual Healthcare By:*

- |                      |            |
|----------------------|------------|
| 1. Eshetu Girma      | ETS0255/12 |
| 2. Kirubel Tadesse   | ETS1007/12 |
| 3. Kibrom Hailu      | ETS0399/12 |
| 4. Gebrehiwot Tesfye | ETS0310/12 |
| 5. Lydia Abraham     | ETS1016/12 |

*Submitted to Addis Ababa Science and Technology University for the award of Bachelor of Science degree in Electrical and Computer Engineering (Computer Engineering focus area)*

June, 2024

Addis Ababa, Ethiopia

## Examining Committee Approval Sheet

Title of Thesis:

Student Names and ID:

Approved by the examining committee members:

| Name | Academic Rank | Signature | Date |
|------|---------------|-----------|------|
|------|---------------|-----------|------|

Advisor :

Co-Advisor:

Examiner :

Examiner :

| Name | Signature | Date |
|------|-----------|------|
|------|-----------|------|

DC Chairperson :

Associate Dean for:

Undergraduate Programs

## **ACKNOWLEDGMENT**

First, we would like to thank Almighty God for his guidance up to this day. We express our heartfelt gratitude to our advisor, Dr. Ashenafi Yadessa (Ph.D.), who has been guiding us throughout the project. Finally, we would like to thank the entire AASTU community, both directly and indirectly, for their contributions and assistance in our project.

## **ABSTRACT**

The healthcare industry has been evolving since the discovery of the microscope, along with numerous technological advancements globally. Likewise, Ethiopia can benefit immensely from up-to-date technological platforms and emerging technologies. According to Peter Webster (2020), during the 2019 COVID-19 era, virtual sessions have improved efficiency.[1] Harnessed since the global pandemic, virtual healthcare centers can provide numerous advantages, including cost-effectiveness, affordability, efficient workflow, and accessibility.

This project aims to implement a virtual healthcare center utilizing emerging technologies in healthcare services in Ethiopia. Currently, it is limited to diagnosing 14 skin and eye diseases, with features that provide efficient accessibility for the limited human resources in this specialized medical area in Ethiopia. The system assists local physicians in rural areas to make accurate diagnoses and allows for referrals to specialist physicians in cases of severe diagnoses.

Using analysis and observation, along with guidance from healthcare professionals, a system has been developed. This system consists of a web application platform with rich features. The project is composed of two machine learning models. This composition ensures high-accuracy diagnoses and simplifies further diagnosis in severe medical cases.

**Keywords:** Digital health platforms, Technology and Healthcare

# Contents

|                                                    |            |
|----------------------------------------------------|------------|
| <b>Acknowledgements</b>                            | <b>ii</b>  |
| <b>Abstract</b>                                    | <b>iii</b> |
| <b>List of Abbreviations</b>                       | <b>1</b>   |
| <b>Chapter One</b>                                 | <b>2</b>   |
| <b>1 INTRODUCTION</b>                              | <b>2</b>   |
| 1.1 Background of The Study . . . . .              | 3          |
| 1.2 Statement of The Problem . . . . .             | 3          |
| 1.3 Objective of the study . . . . .               | 4          |
| 1.3.1 General objective of the study . . . . .     | 4          |
| 1.3.2 Specific objective of the study . . . . .    | 4          |
| 1.4 Scope and Limitations of the study . . . . .   | 4          |
| 1.4.1 Scope of the study . . . . .                 | 4          |
| 1.4.2 Limitations of The Study . . . . .           | 5          |
| 1.5 Significance of The Study . . . . .            | 5          |
| 1.6 Outline of The Research Project . . . . .      | 6          |
| <b>Chapter Two</b>                                 | <b>6</b>   |
| <b>2 LITERATURE REVIEW</b>                         | <b>7</b>   |
| 2.1 Theoretical Review of The Literature . . . . . | 7          |
| 2.2 Related Works . . . . .                        | 9          |
| 2.3 Research Gaps . . . . .                        | 11         |
| <b>Chapter Three</b>                               | <b>11</b>  |

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| <b>3</b> | <b>METHOD and DESIGN</b>                                  | <b>12</b> |
| 3.1      | Method . . . . .                                          | 12        |
| 3.1.1    | Data Collection Methods . . . . .                         | 13        |
| 3.1.2    | Significance and Uniqueness . . . . .                     | 13        |
| 3.2      | Design . . . . .                                          | 14        |
| 3.2.1    | Design Overview . . . . .                                 | 14        |
| 3.2.2    | Website Development . . . . .                             | 15        |
| 3.2.2.1  | System Design and Architecture: . . . . .                 | 15        |
| 3.2.2.2  | Front-end(React JS): . . . . .                            | 18        |
| 3.2.2.3  | Back-end(Django and PostgreSQL): . . . . .                | 24        |
| 3.2.2.4  | Database Design (Class and ER Diagram) . . . . .          | 24        |
| 3.2.2.5  | API Design (Restful API in Django, Djoser): . . . . .     | 27        |
| 3.2.2.6  | Model Integration(ML Integration with Back-end) . . . . . | 27        |
| 3.2.3    | Machine Learning Development . . . . .                    | 30        |
| 3.2.3.1  | Data Acquisition . . . . .                                | 30        |
| 3.2.3.2  | Data Preprocessing . . . . .                              | 36        |
| 3.2.3.3  | Model Building with Resnet-50 . . . . .                   | 40        |
| 3.2.3.4  | Model Testing and Evaluation . . . . .                    | 48        |
| 3.2.4    | Integration and Deployment . . . . .                      | 50        |
| 3.2.4.1  | Database and Backend Integration . . . . .                | 50        |
| 3.2.4.2  | Back-end and Front-end Integration: . . . . .             | 51        |
| 3.2.5    | Testing and Deployment Documentation . . . . .            | 52        |
|          | <b>Chapter Four</b>                                       | <b>52</b> |
| <b>4</b> | <b>RESULT and DISCUSSIONS</b>                             | <b>53</b> |
| 4.1      | Result . . . . .                                          | 53        |
| 4.1.1    | Developed Web Application . . . . .                       | 54        |
| 4.1.2    | Trained Machine Learning Model . . . . .                  | 55        |
| 4.2      | Discussions . . . . .                                     | 56        |
|          | <b>Chapter Five</b>                                       | <b>56</b> |

|                                        |           |
|----------------------------------------|-----------|
| <b>5 CONCLUSION and RECOMMENDATION</b> | <b>57</b> |
| 5.1 Conclusion . . . . .               | 57        |
| 5.2 Recommendations . . . . .          | 57        |
| <b>References</b>                      | <b>58</b> |
| <b>Appendix</b>                        | <b>60</b> |

# List of Figures

|      |                                                       |    |
|------|-------------------------------------------------------|----|
| 3.1  | Water Fall SDLC Diagram . . . . .                     | 12 |
| 3.2  | Machine Learning Workflow . . . . .                   | 13 |
| 3.3  | Activity Diagram . . . . .                            | 16 |
| 3.4  | Use Case Diagram . . . . .                            | 17 |
| 3.5  | Activity Diagram . . . . .                            | 18 |
| 3.6  | User Interface . . . . .                              | 21 |
| 3.7  | Class Diagram . . . . .                               | 25 |
| 3.8  | ER Diagram . . . . .                                  | 26 |
| 3.9  | Code Snippet . . . . .                                | 28 |
| 3.10 | Some APIs used . . . . .                              | 29 |
| 3.11 | HAM10000 Dataset based on age . . . . .               | 31 |
| 3.12 | Skin Cancer Vulnerable Body Parts . . . . .           | 32 |
| 3.13 | Gender Bias for different skin cancer types . . . . . | 33 |
| 3.14 | Cell Type Frequency . . . . .                         | 33 |
| 3.15 | Distribution based on Age . . . . .                   | 35 |
| 3.16 | Distribution based on Gender . . . . .                | 35 |
| 3.17 | Code Snippet checking for Null values . . . . .       | 36 |
| 3.18 | Code Snippet checking for Null values . . . . .       | 36 |
| 3.19 | Preprocessing code snippet . . . . .                  | 40 |
| 3.20 | Architecture of ResNet-50 . . . . .                   | 42 |
| 3.21 | Layers of Trained Model for Eye Disease . . . . .     | 45 |
| 3.22 | Code Snippet showing Trained Layers . . . . .         | 45 |
| 3.23 | Layers of Trained Model for Skin Disease . . . . .    | 47 |
| 3.24 | Code Snippet used to Evaluate the Model . . . . .     | 48 |
| 3.25 | Out put for the evaluation . . . . .                  | 48 |



|      |                                                                 |    |
|------|-----------------------------------------------------------------|----|
| 3.26 | Model Accuracy and Model Loss representation in graph . . . . . | 49 |
| 3.27 | Confusion Matrix . . . . .                                      | 49 |
| 3.28 | Model Accuracy . . . . .                                        | 50 |
| 3.29 | Model Loss . . . . .                                            | 50 |
| 4.1  | Deployed Website . . . . .                                      | 55 |
| 1    | Importing Libraries and Reading Data . . . . .                  | 61 |
| 2    | Creating ResNet50 Model for Eye Diagnosis . . . . .             | 61 |
| 3    | Training the Model for Eye Diagnosis . . . . .                  | 62 |
| 4    | Model Evaluation . . . . .                                      | 62 |
| 5    | Creating ResNet50 Model for Eye Diagnosis . . . . .             | 62 |
| 6    | Training the Model for Eye Diagnosis . . . . .                  | 62 |
| 7    | Model Evaluation . . . . .                                      | 63 |
| 8    | Training the Model for Eye Diagnosis . . . . .                  | 63 |
| 9    | Cell Type Frequency . . . . .                                   | 63 |
| 10   | Skin Disease Model Summary Output . . . . .                     | 64 |
| 11   | Skin Disease Model Test on Training Data . . . . .              | 64 |
| 12   | Skin Disease Model Test on Test Data . . . . .                  | 65 |
| 13   | Eye Disease Model Summary Output . . . . .                      | 65 |
| 14   | Localization Area Frequencies . . . . .                         | 66 |
| 15   | The Skin Data Set Partially . . . . .                           | 66 |
| 16   | Eye Output . . . . .                                            | 67 |
| 17   | Eye Output . . . . .                                            | 67 |

## **LIST OF ABBREVIATIONS**

ML - Machine Learning

API - Application Programming Interface

ER - Entity Relationship

SDLC - Software Development Life Cycle

HAM10000 - Human Against Machine with 10,000 training images

# **Chapter 1**

## **INTRODUCTION**

Ethio Virtual Healthcare is a digital platform specializing in skin and eye diseases. Its primary goal is to provide high-quality, affordable, timely, and accurate diagnoses, ensuring efficient human resource utilization. These services are accessible to patients regardless of their geographical location. The platform addresses the challenges of limited access to specialized healthcare, particularly in remote or underserved areas, by leveraging digital technology to connect patients with expert medical professionals.

The platform facilitates clinical recording and offers AI-powered diagnostics to aid in the accurate assessment of skin and eye diseases. Depending on the severity of the case, it also provides a referral option to specialists. This approach ensures that patients receive timely and appropriate care while maintaining legal and professional standards within the healthcare system.

## **1.1 Background of The Study**

The healthcare system in Ethiopia faces numerous challenges that hinder the provision of equitable and efficient healthcare services. Despite various initiatives aimed at improving healthcare delivery, several barriers remain persistent. Emerging technologies and digital solutions can significantly be utilized to make a difference.

Intrigued by the COVID-19 Pandemic, telemedicine has emerged as the most efficient way of engaging with patients without the risk of vulnerability. According to Perrone, Telemedicine will be required to streamline patients' appointments and reduce overcrowding and healthcare expenses, making it affordable and accessible and hence more sustainable, with major benefits.[2]

Technological intervention in the healthcare plays an important role. Emerging technologies can be integrated to improve Ethiopia's healthcare industry's efficiency. Healthcare access in Ethiopia's rural areas is full of challenges. According to a study, the financial burden of health care is exacerbated by the less privileged populations: the uninsured, the chronically ill, the elderly, and the socially excluded.[3]

This project aims to address the challenges of efficient utilization of specialist physicians, affordability, and access to healthcare through an AI-enabled platform. This platform will streamline processes to support local physicians in rural areas, thereby enhancing healthcare delivery and ensuring better health outcomes for underserved populations.

## **1.2 Statement of The Problem**

The healthcare system in many developing countries faces significant challenges in providing equitable and efficient services. These challenges often stem from limited financial resources, inadequate infrastructure, and human resource constraints. To address these issues, technological interventions offer a promising solution by enhancing the efficient utilization of human resources, overcoming physical barriers, and reducing non-medical costs.

The development of an AI-powered digital health platform that assists in the timely diagnosis and referral of severe cases is further explored.

### **1.3 Objective of the study**

The objective of this study is to use emerging technologies to improve the efficiency of the healthcare industry in Ethiopia.

#### **1.3.1 General objective of the study**

To develop an AI-enabled software system that can assist health professionals in the diagnosis of skin and eye disease.

#### **1.3.2 Specific objective of the study**

Achieving these specific objectives, the study aims to demonstrate the transformative potential of emerging technologies in addressing the systemic challenges faced by the healthcare industry in Ethiopia.

1. To conduct in depth analysis of the problem
2. To study different Machine Learning Models
3. To build skin and eye diagnosing classification ML Models
4. To Identify system requirements
5. To Design the software system
6. To Implement the software
7. To Integrate the Machine Learning Model with Web app.
8. To Validate

### **1.4 Scope and Limitations of the study**

#### **1.4.1 Scope of the study**

This study aims to explore the potential of such technologies specifically in the context of skin and eye diseases. By narrowing the focus to these two areas, the study seeks to provide a detailed analysis of how technological interventions can enhance the efficiency and accessibility of healthcare services.

The project is bounded with the study of available Machine Learning Models and implementation of customization with different data sets. The software is limited to diagnosis but not other healthcare management systems.

Skin and eye diseases are prevalent health issues that significantly impact individuals' life quality. These conditions often require specialized care, which is scarce in many parts of Ethiopia, particularly in rural areas. Skin diseases can range from common conditions like eczema and psoriasis to more severe infections and dermatological disorders. Similarly, eye diseases, such as cataracts, glaucoma, and trachoma, are major causes of visual impairment and blindness. Addressing these conditions effectively requires timely diagnosis and treatment by specialists, who are often concentrated in urban centers.

One of the primary objectives of this study is to enhance the diagnostic and treatment capabilities for skin and eye diseases through the use of AI-enabled platforms. This is particularly beneficial for skin and eye conditions, where visual assessment and follow-up are critical components of effective treatment.

The platform can further augment the diagnostic process by analyzing medical images and providing preliminary diagnostic suggestions. Separately trained AI algorithms can identify various skin conditions from photographs and detect signs of eye diseases from fundus copy retinal scans. These tools can support healthcare professionals in making accurate diagnoses and recommending appropriate treatments, thereby improving the overall quality of care.

#### **1.4.2 Limitations of The Study**

The project's focus on eye and skin disease specializations is primarily due to the limited availability of training data for other conditions. Additionally, the platform's accessibility to patients is restricted by legal and standards issues within the healthcare system. Furthermore the limited with scope of services the project excludes billing and e-pharmacy services along with other hospital management platform features.

#### **1.5 Significance of The Study**

The healthcare system in Ethiopia faces numerous challenges, including limited financial resources, inadequate infrastructure, and a shortage of specialized professionals. These issues

are particularly acute in the fields of dermatology and ophthalmology, where the demand for specialized care far exceeds the available supply. The significance of this study lies in its potential to leverage emerging technologies to address these challenges, thereby, transforming healthcare delivery for skin and eye diseases in Ethiopia.

One of the most significant contributions of this study is its potential to enhance healthcare accessibility, particularly for underserved populations in rural areas. Geographic barriers and the concentration of specialists in urban centers limit access to quality healthcare for many Ethiopians. Through Tele-health services and mobile health (mHealth) applications, the study aims to overcome these barriers by enabling remote consultations and continuous healthcare support. This not only improves access to specialized care but also ensures that patients receive timely and accurate medical advice, regardless of their location.

Efficient utilization of healthcare resources is another critical aspect of this study. Ethiopia faces a shortage of specialized healthcare professionals, and optimizing the use of existing resources is essential to meet the growing demand for dermatological and ophthalmological care. AI-enabled platforms can support healthcare professionals by providing diagnostic assistance and streamlining patient management processes. This allows specialists to focus on more complex cases and improves the overall efficiency of healthcare delivery. By enhancing the productivity of healthcare workers, the study contributes to a more effective use of limited resources.

## **1.6 Outline of The Research Project**

The first chapter provides the background and context for the study of Ethio-Virtual Healthcare. It states the problem being addressed and outlines the general and specific objectives of the research project. The second chapter presents a theoretical examination of relevant research on virtual healthcare systems. It discusses related works and projects in this domain, and identifies key research gaps that the current study aims to address. The third chapter describes the methodological approach and system design employed in this project. On the fourth chapter the key results of the project, including the developed web application and the trained machine learning model are presented. The final chapter summarizes the main conclusions drawn from the study. Additionally, recommendations are provided for future work and potential improvements to the Ethio-Virtual Healthcare system going forward.

# Chapter 2

## LITERATURE REVIEW

The following chapter presents a comprehensive review of the existing literature on the subject matter, synthesizing key findings and insights to build a compelling argument for the significance and relevance of the proposed project.

### 2.1 Theoretical Review of The Literature

The healthcare system in Ethiopia faces numerous challenges that hinder the provision of equitable, affordable, and efficient healthcare services. Despite various initiatives aimed at improving healthcare delivery, several barriers remain persistent. These challenges can be broadly categorized into financial, infrastructural, procedural, and human resource-related issues.

**Financial Barriers:** a significant challenge in Ethiopia's healthcare sector is the low health-care expenditure. Compared to the World Health Organization (WHO) average for low-income countries (LICs), Ethiopia's healthcare spending is markedly insufficient. This underfunding is attributed to low government budgets, high levels of corruption, and inadequate health insurance coverage. Specifically, the low enrollment and high dropout rates from Community-Based Health Insurance (CBHI) programs exacerbate the financial strain on the healthcare system. Furthermore, high out-of-pocket (OOP) payments burden poor households, making healthcare services unaffordable for many.[4]

**Infrastructural and Supply Chain Issues:** infrastructural deficiencies are another critical challenge. Many healthcare facilities in Ethiopia lack the necessary physical infrastructure and medical supplies. Shortages of drugs and medical supplies, combined with inadequate di-



agnostic capacity, impede the delivery of quality healthcare services. Additionally, poor healthcare infrastructures contribute to service disparities, where rural and remote areas are disproportionately affected.[5]

**Human Resource Challenges:** the healthcare workforce in Ethiopia is plagued by several issues, including a high turnover rate, absenteeism, and poor motivation among healthcare professionals. The absence of performance-based reward systems and inadequate support for healthcare workers contribute to low morale and inefficiency. Furthermore, the lack of trained personnel, especially specialists, and inadequate training for existing staff hinder the delivery of specialized care, such as for eye and skin diseases.[5]

Globally, healthcare systems face numerous challenges that hinder their ability to provide timely, accurate, and affordable services to patients. These challenges are multifaceted, encompassing issues related to economic disparities, resource allocation, technological integration, and ethical practices.[6]

Recently, technological intervention has been utilized especially in the case of a universal pandemic COVID-19. The integration of technology in healthcare presents numerous opportunities to enhance the efficiency, accuracy, and accessibility of health services. This transformation is driven by the need for effective health communication, which is fundamental in establishing a strong, empathetic relationship between health professionals and their patients. Historically, health communication relied heavily on traditional methods such as signage and public education campaigns. However, the advent of digital technology has revolutionized how health information is disseminated and consumed.

One of the most significant opportunities presented by technology in healthcare is the improvement of health communication. Digital platforms, including websites and social media, allow health institutions to disseminate medical and scientific knowledge quickly and broadly. These platforms can present information in a user-friendly manner, making it accessible to diverse population groups. This shift from traditional, one-way communication to a more interactive, bidirectional exchange of information empowers patients to seek out and engage with health information actively.

Technology also enhances the reach of healthcare services, particularly through the use of

telemedicine and mobile health (mHealth) applications. Telemedicine allows patients in remote or underserved areas to access specialist care without the need for travel, breaking down geographical barriers. mHealth apps provide continuous support and health monitoring, making healthcare more accessible and convenient. These technological interventions ensure that high-quality healthcare services can reach a broader audience, regardless of their location.[7]

Information and Communication Technology (ICT) holds significant potential to transform various aspects of the healthcare system, addressing critical challenges and enhancing the quality of health services. This potential is particularly impactful in developing countries, where the healthcare infrastructure often faces considerable burdens of disease and a shortage of skilled health professionals.[8]

In 2006, the physician-to-population ratios in Amhara, Oromia, and SNNPR regional states were found to be 1:280,000, 1:220,000, and 1:230,000, respectively. During the last 23 years, the physician deficit analysis of the WHO benchmark for poor countries (1:10,000) found the lowest record at the national and regional levels in the previous 12 years.[9] In recent years this target number has been achieved. WHO reported Ethiopia had 1.08 physicians to 10,000 population ratio in 2022.[10]

Despite the increasing number of healthcare professionals, the limited healthcare facilities hinder their contribution to the country's well-being. Additionally, the number of specialists, particularly in eye and skin diseases, remains insufficient. However, by leveraging emerging technologies, it is possible to make these limited human resources accessible to a vast population without the constraint of physical distance.

## **2.2 Related Works**

ICTs are being increasingly integrated into healthcare systems in both developing and developed countries in ways that were unimaginable just 20 years ago. This has enabled ubiquitous and spatiotemporal models of healthcare delivery where patients and providers do not need to be physically co-located.

Mobile health (mHealth) approaches leveraging mobile phones are emerging as a promising tool to improve access to healthcare in developing countries. mHealth solutions can help

address challenges like shortages of healthcare workers and the rise of chronic diseases in developing countries. Studies have found mHealth interventions to be effective for improving outcomes in areas like diabetes management, medication adherence for cardiovascular disease, and depression treatment. However, there are still barriers to widespread adoption of mHealth in developing countries that need to be addressed. Telemedicine and mHealth are considered 'game changers' with huge potential to transform healthcare delivery in developing countries, but many countries have been slow to implement these technologies. In the long run, telemedicine can reduce the overall costs of healthcare delivery and access in developing countries, despite high upfront costs for technology and training. As ICTs continue advancing, there are exciting possibilities on the horizon like autonomous drones delivering medical supplies and even reasoning about diagnoses and prescriptions.[11]

Existing studies exploring the implementation of Tele-intensive care consultation services, particularly in the preliminary stage of the Virtual Hospital North Rhine-Westphalia (VKh), highlight significant benefits in terms of patient care and individual patient outcomes. The evidence-based added value of Tele-intensive care has been demonstrated, showcasing its potential to address challenges such as resource shortages and facilitate citizen-centered care. However, these studies may lack a comprehensive analysis of the scalability and generalization of tele-intensive care networks.[12]

The Aachen Expert Center's report on telemedically co-managed COVID-19 patients within the Virtual Hospital's preliminary stage demonstrates high-quality care outcomes, notably in ventilated severely ill patients. The study emphasises a lethality rate of 34.2% and a transfer rate of 8%, showcasing the effectiveness of tele-intensive medical consultation services. However, there might be a lack of exploration into the patient experience, including factors such as satisfaction and engagement.

## **2.3 Research Gaps**

Existing studies emphasise the benefits of tele-intensive care consultation services, there is a need for further investigation into the scalability and broader applicability of these services. Unanswered questions include the adaptability of tele-intensive care networks across different healthcare settings and their impact on diverse patient populations.

Undermining the advantages of the Virtual Hospital NEWs telemedical network, there is a gap in understanding the long-term sustainability and effectiveness of this structure. Further research is needed to explore the scalability and ongoing impact of telemedical networks, especially in diverse healthcare settings.

While the reported outcomes highlight the clinical effectiveness of tele-intensive medical consultation services, there is a research gap in understanding the patient experience. Further investigation into patient satisfaction, engagement, and the holistic impact of tele-intensive care on the overall healthcare experience is warranted.

In the literature review, we have firmly established the importance and potential of digitizing healthcare systems, particularly in developing countries, while also highlighting existing gaps and challenges. The next chapter will delve into the specific methods and approaches that will be used to implement the proposed digital health solution, building upon the insights gained from this comprehensive literature review.

# Chapter 3

## METHOD and DESIGN

### 3.1 Method

The project employs various methods to develop a comprehensive understanding of the problem domain and effectively implement the proposed system. First, an in-depth literature review was conducted to gain insights from prior research and established methodologies. Additionally, the well-recognized waterfall software development life cycle model was systematically applied throughout the project's progression.

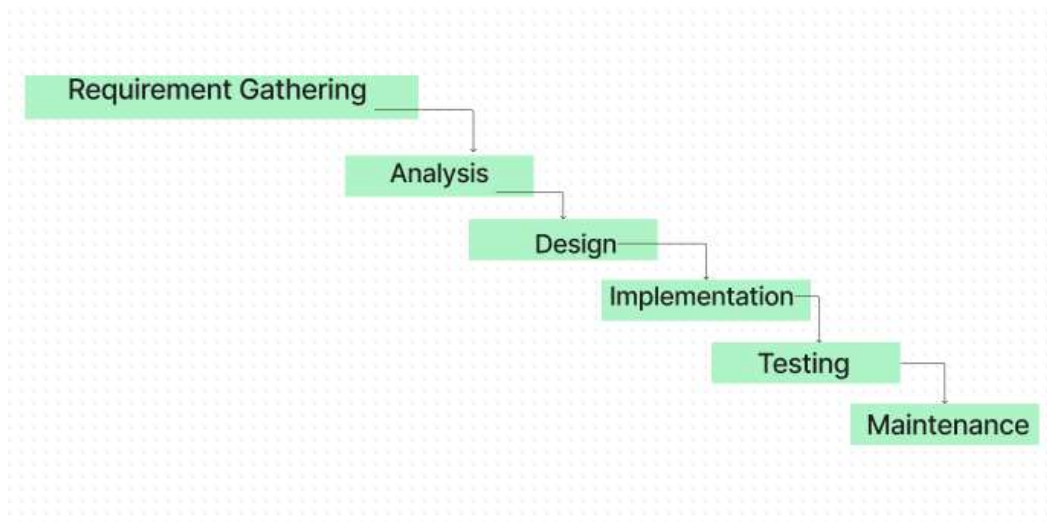


Figure 3.1: Water Fall SDLC Diagram

Furthermore, the implementation of machine learning capabilities and integration within the system required in the application of iterative machine learning methodology, detailed in the following sections.

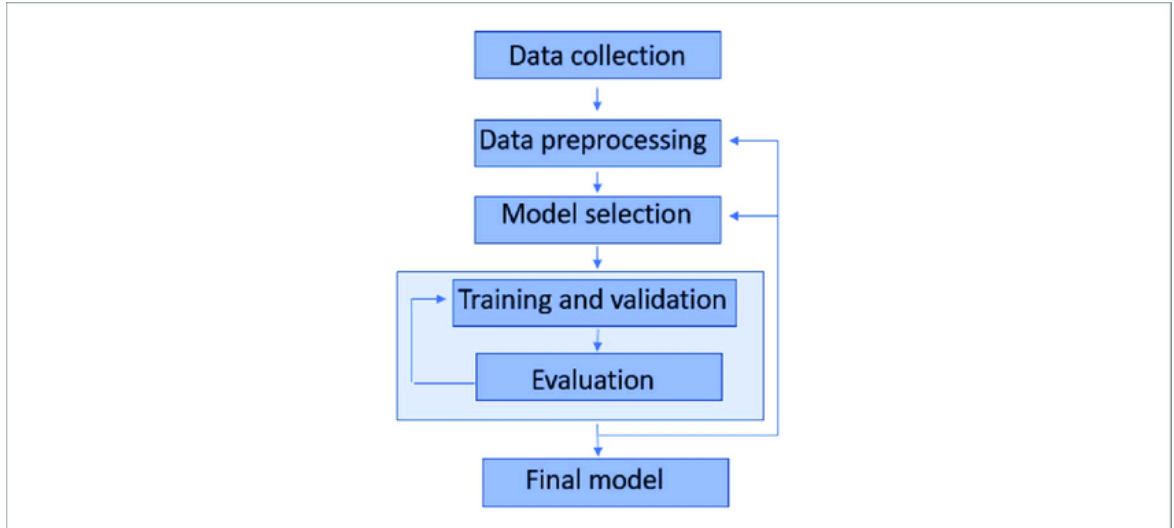


Figure 3.2: Machine Learning Workflow

### 3.1.1 Data Collection Methods

The dataset used to train the AI model was sourced from the following sources:

1. Eye Dataset: *Ocular Disease Recognition* [13]
2. Skin Dataset: *Skin Cancer MNIST: HAM10000* [14]

### 3.1.2 Significance and Uniqueness

The approaches utilized highlights the significance of considering legal standards in health-care when designing remote healthcare platforms. This includes ensuring compliance with regulations and standards that govern healthcare services. Additionally, the study reveals the importance of simplifying user interface features to enhance the end-user experience. This includes designing interfaces that are intuitive, user-friendly, and accessible to a wide range of users, including those with disabilities.

The socioeconomic advantages of the platform are vast. By ensuring access to remote locations, the platform can bridge the healthcare gap in underserved areas, providing timely and affordable access to specialized healthcare services. This can lead to improved health outcomes and reduced healthcare costs. The efficient distribution of specialized human resources concentrated in urban areas can also lead to better resource allocation and utilization.

Furthermore, the simplified procedures and prioritization of severe cases can ensure that patients receive the necessary care in a timely manner, reducing the risk of adverse outcomes.

## **3.2 Design**

### **3.2.1 Design Overview**

The Ethio Virtual Healthcare web application platform is designed to connect local physicians in remote rural areas with specialists for severe cases of referral. The platform utilizes a combination of React, Django, and machine learning (ML) technologies to achieve its goals. The architecture of the platform consists of several key components:

The key components involved in the design of the platform include:

1. **User Interface:** The user interface is built using React and provides a seamless and intuitive experience for both local physicians and specialists.
2. **Server-Side Logic:** The server-side logic is handled by Django, which provides a robust and scalable infrastructure for the platform.
3. **Machine Learning Models:** The ML models are trained on a large dataset of medical images and are designed to identify specific patterns and features that are indicative of various medical conditions.
4. **Database:** The platform uses a database to store patient data, medical images, and other relevant information.
5. **API Integrations:** The platform integrates with various APIs to access medical databases, patient records, and other relevant information.

React handles the user interface and user input, while Django handles the server-side logic and database interactions. The ML models are integrated with Django to provide real-time analysis and recommendations. This integration enables the platform to provide a seamless and efficient user experience, while also ensuring that the diagnosis process is accurate and reliable.

### **3.2.2 Website Development**

#### **3.2.2.1 System Design and Architecture:**

the high-level architecture of the Ethio Virtual Healthcare web application platform consists of the following key components:

1. React-based Front-end
  - (a) Responsible for the user interface and user interactions.
  - (b) Communicates with the Django back-end through APIs.
2. Django-based Back-end
  - (a) Handles the server-side logic, database interactions, and API integrations.
  - (b) Provides the necessary APIs for the React frontend to interact with the platform.
3. Database
  - (a) Stores patient records, medical images, and other relevant information.
  - (b) Ensures secure and reliable data storage and retrieval.
4. API Integrations
  - (a) Enables the platform to integrate with external healthcare systems and data sources.
  - (b) Facilitates seamless data exchange and collaboration.



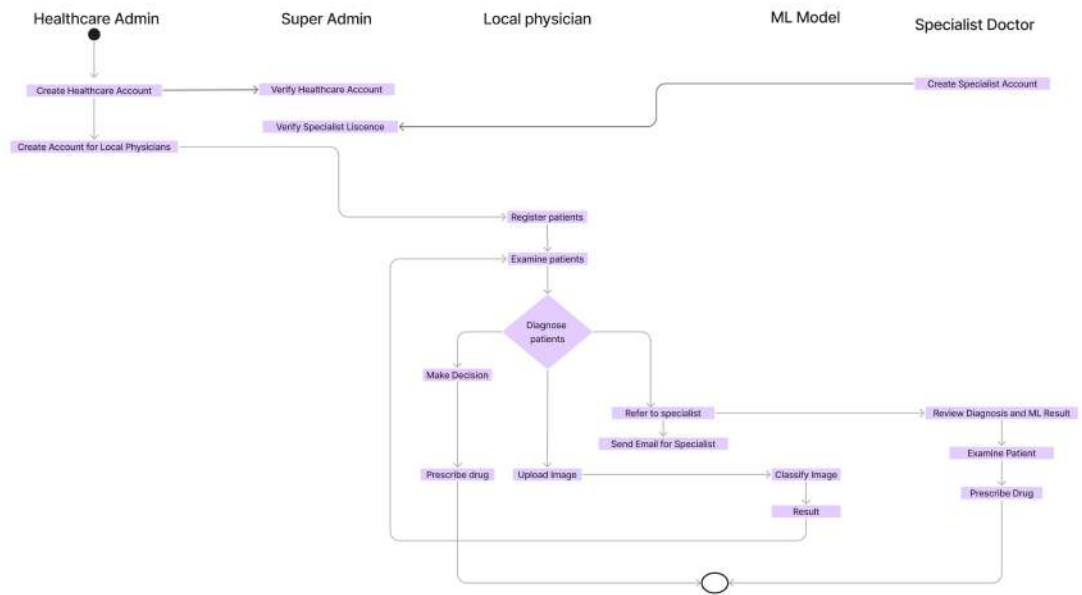


Figure 3.3: Activity Diagram

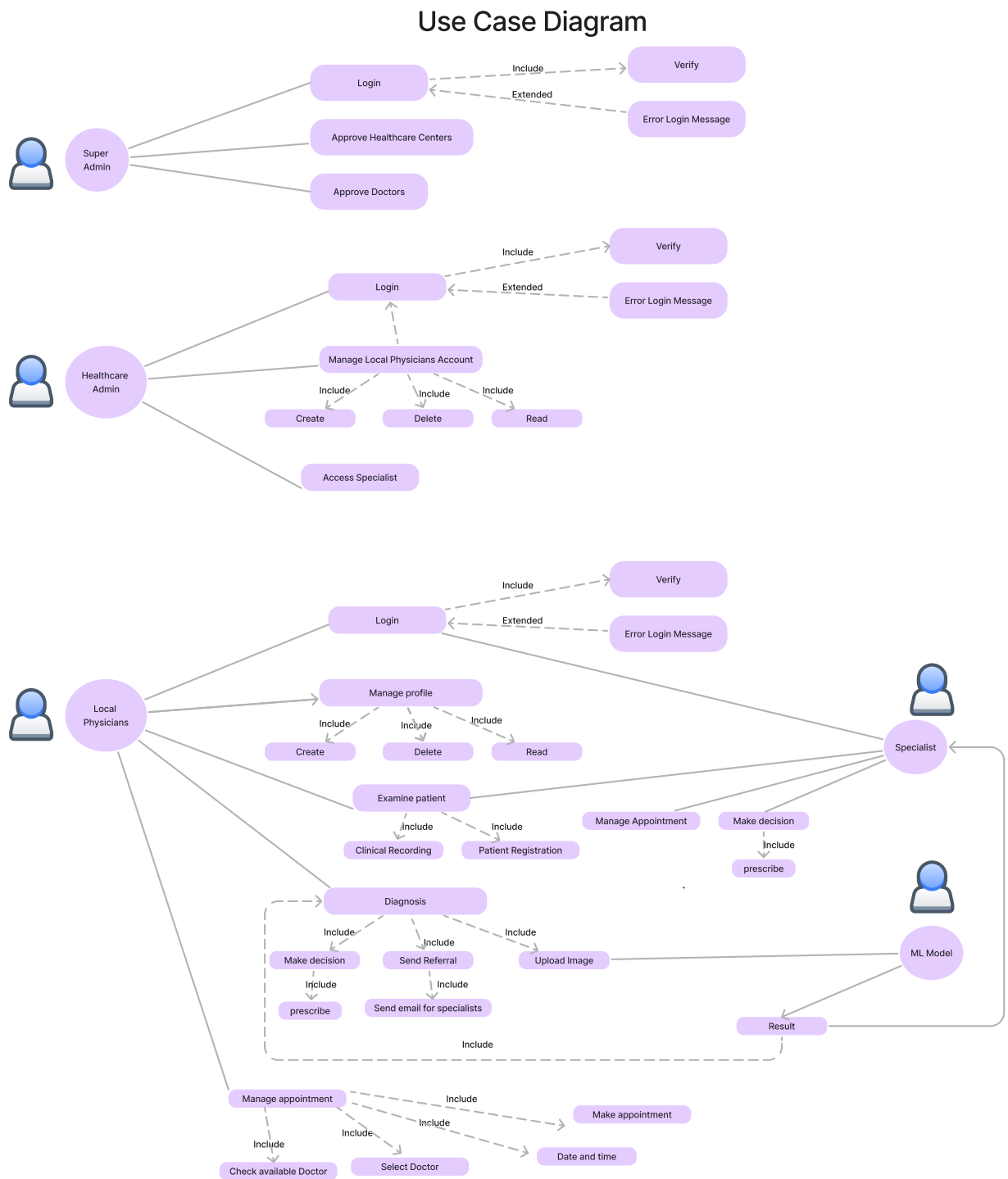


Figure 3.4: Use Case Diagram

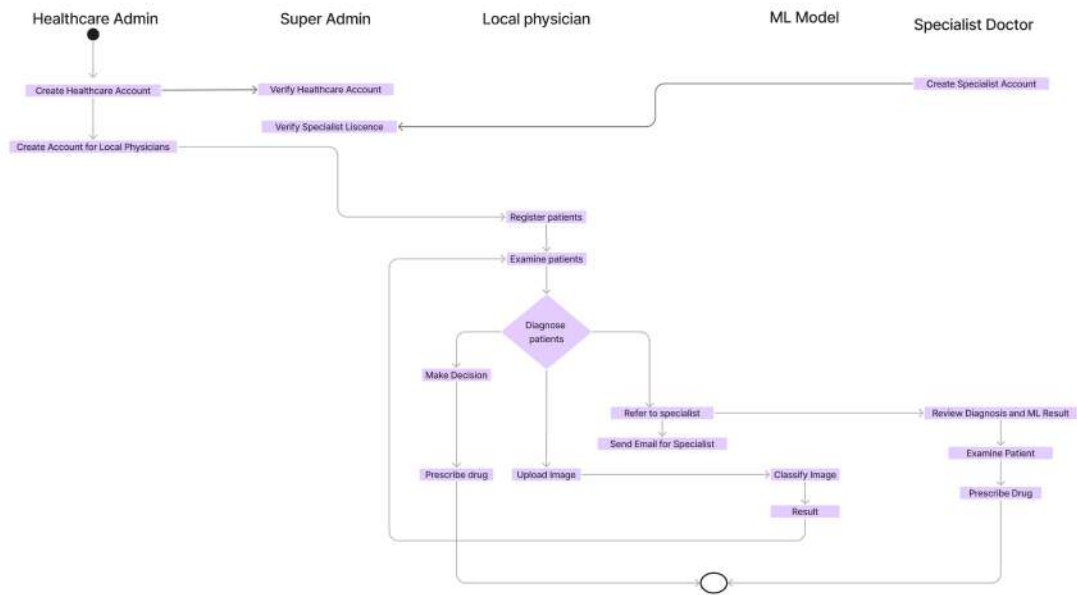


Figure 3.5: Activity Diagram

### 3.2.2.2 Front-end(React JS):

react JS, usually referred to simply as React, is a free and open-source JavaScript library created by Meta (formerly Facebook) for building user interfaces (UI) of web applications. It is known for its component-based architecture, which allows developers to create reusable UI elements.

**Rationale:** the decision to implement React as the front-end framework was informed by several key factors. Firstly, React’s component-based architecture promotes code reusability and maintainability. This modular approach facilitates the efficient development and management of complex user interfaces (UIs). Secondly, React boasts a rich ecosystem of supporting libraries and tools. This extensive ecosystem empowers developers to leverage pre-built components and functionalities, accelerating the development process. Ultimately, the combination of React’s inherent flexibility, powerful features, and a familiar development environment for the team solidified its selection as the frontend framework for this project.

**Development Environment (vite):** vite is a modern build tool that enhances the development experience for web projects by leveraging native ES (ECMAScript) modules.

**Key Features of Vite:**

1. Fast Development Server: vite starts the development server quickly, providing a responsive development experience.
2. Hot Module Replacement (HMR): allows modules to update instantly without a full page reload, making development smoother and more efficient.
3. Optimized Builds: produces optimized, performant build outputs using Rollup, ensuring a lightweight final application.
4. ES Module Support: utilizes native ES modules, reducing the need for complex bundling during development and aligning with modern JavaScript standards.
5. Rich Plugin Ecosystem: supports a variety of plugins for different frameworks and tools, making it adaptable to various project needs.

## **User Interface**

1. **Component-Based Architecture:** react's component-based architecture allows us to build encapsulated components that manage their own state, making the development process more efficient and the codebase more maintainable.
2. **Declarative UI:** react enables us to design simple views for each state in our application, and React will efficiently update and render the right components when the data changes.
3. **Performance:** with React's virtual DOM, we can ensure high performance for complex user interfaces, making our application responsive and fast.
4. **Libraries and Tools:** we used libraries like Ant Design for a comprehensive set of UI components that follow modern design principles, and React Icons for a rich collection of icons that enhance the visual appeal of our application.

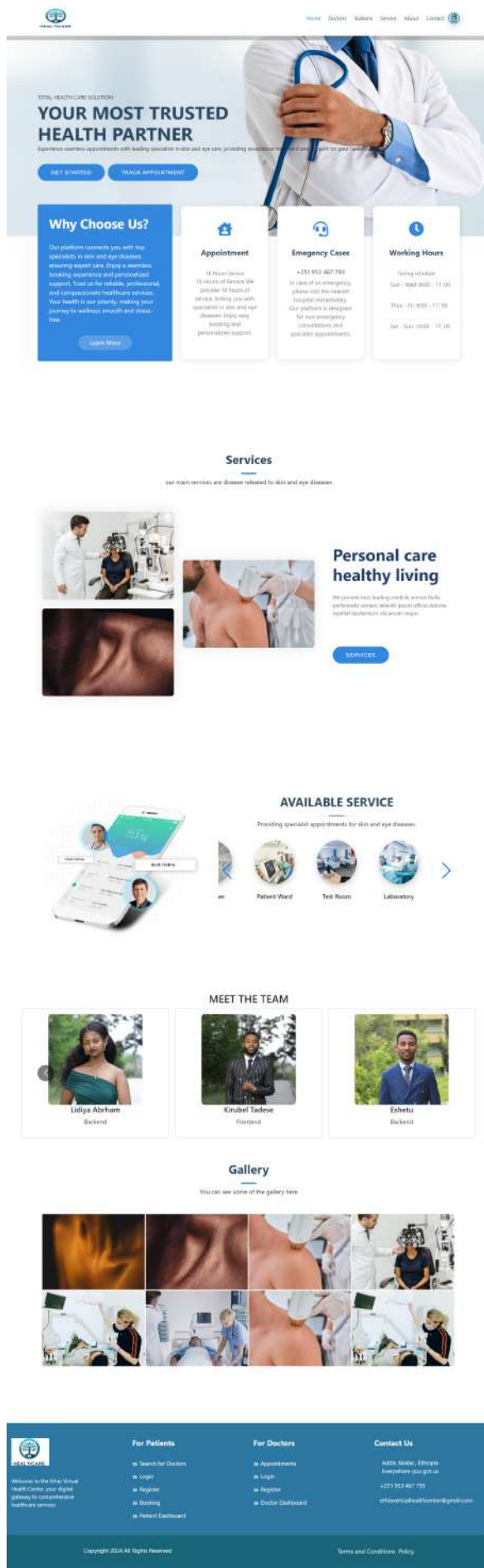


Figure 3.6: User Interface

## **Data Fetching**

1. **State Management:** we use React hooks such as `useState`, `useEffect`, `useLocation`, `useParams`, and `useForm` to manage state and handle side effects efficiently. These hooks enable us to fetch and update data dynamically, ensuring that our UI reflects the latest data.
2. **Asynchronous Operations:** react's support for asynchronous operations is crucial for dynamic applications that need to interact with APIs and update the UI based on real-time data.
3. **Libraries and Tools:** we incorporate various libraries to streamline data fetching and state management, ensuring our application handles asynchronous data interactions smoothly and efficiently.
4. **Error Handling:** implementing robust error handling mechanisms ensures that our application gracefully handles any issues during data fetching, enhancing the user experience.

## **Components**

1. **Reusability:** react components are highly reusable, allowing us to create a library of components that can be used across different parts of the application, reducing duplication and improving maintainability.
2. **Modularity:** each component in React is modular and encapsulated, which means changes in one component do not affect others. This makes it easier to manage and scale the application.
3. **Custom Hooks:** using custom hooks allows us to encapsulate and reuse logic across different components, promoting cleaner and more efficient code.
4. **Libraries and Tools:** utilizing Ant Design provides us with pre-styled components, ensuring a consistent and professional look throughout our application, while react icons enhance the visual elements with high-quality icons.

## **CSS**

1. CSS-in-JS: CSS-in-JS is used in solutions like styled-components or Emotion to keep the concerns of styling and component logic combined in a single location, making the components more self-contained and easier to maintain.
2. Responsive Design: ensuring that our UI is responsive and works well across different devices and screen sizes is crucial. We implement responsive design principles to provide a seamless user experience on mobile, tablet, and desktop devices.



### **3.2.2.3 Back-end(Django and PostgreSQL):**

Django is a high-level web application framework written in Python that adheres to an open-source licensing model. This framework is designed to expedite the development process for web applications.

**Rationale:** the selection of Django and PostgreSQL as the backend infrastructure for this project was guided by several technical considerations.

Firstly, Django, a high-level Python web framework, was chosen for its emphasis on rapid development and well-structured code. This framework promotes a clean and pragmatic design philosophy, facilitating efficient development cycles. Notably, Django's seamless compatibility with Python-based tools and libraries makes it particularly suitable for integration with artificial intelligence (AI) and machine learning (ML) functionalities, which are integral components of this application.

Secondly, PostgreSQL, a powerful open-source object-relational database management system (ORDBMS), was selected for its robust performance and advanced features.

PostgreSQL's support for complex queries and extensive indexing options ensures efficient data retrieval and manipulation, which are crucial aspects for this application. The combined strengths of Django and PostgreSQL provide a solid foundation for the application, offering scalability, robust security measures, and an environment conducive to efficient development.

### **3.2.2.4 Database Design (Class and ER Diagram)**

ER Diagram : the ER diagram depicts the data model of the Ethio Virtual Healthcare web application, including the entities (e.g., users, patients, referrals) and their relationships.

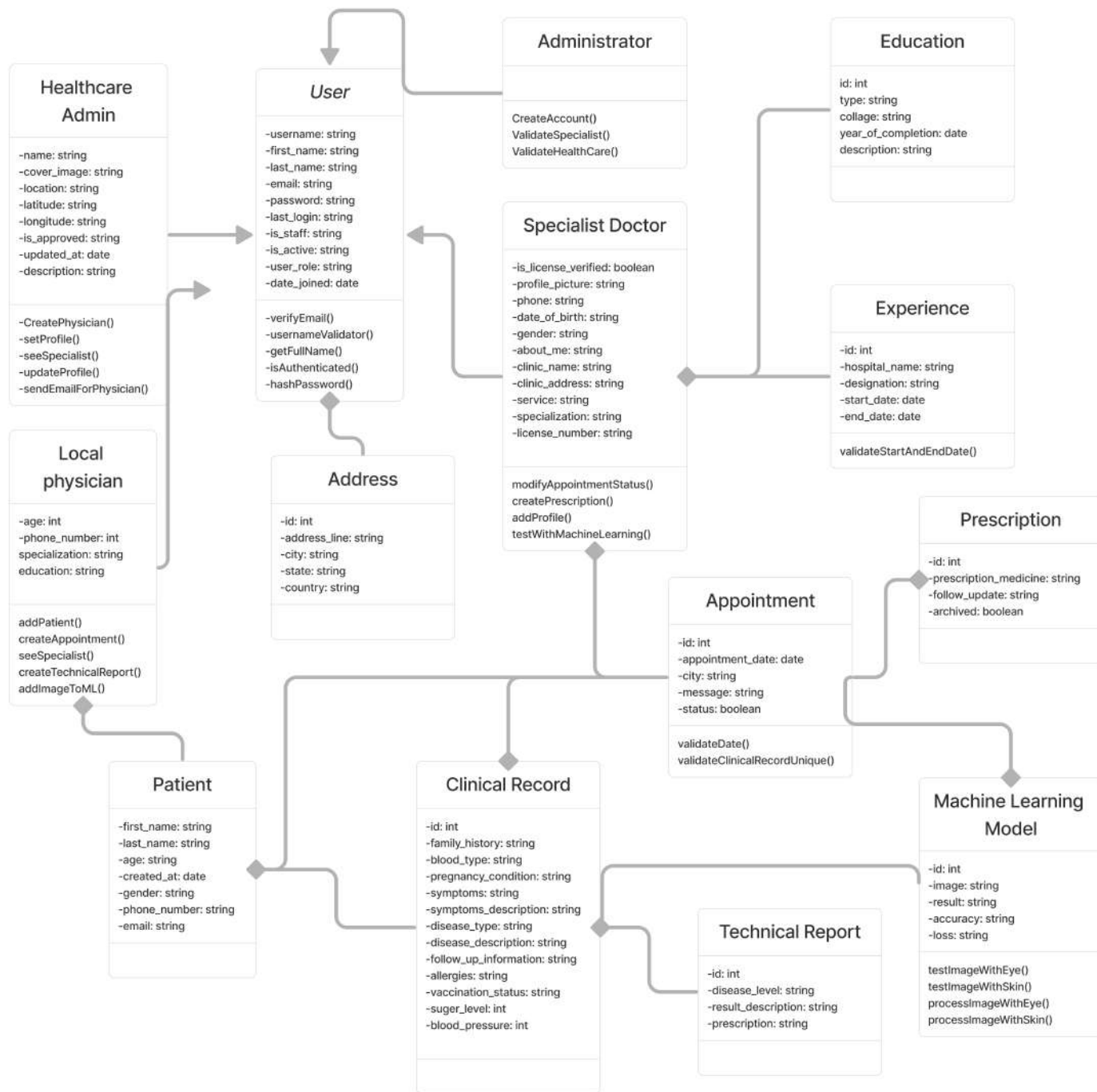


Figure 3.7: Class Diagram

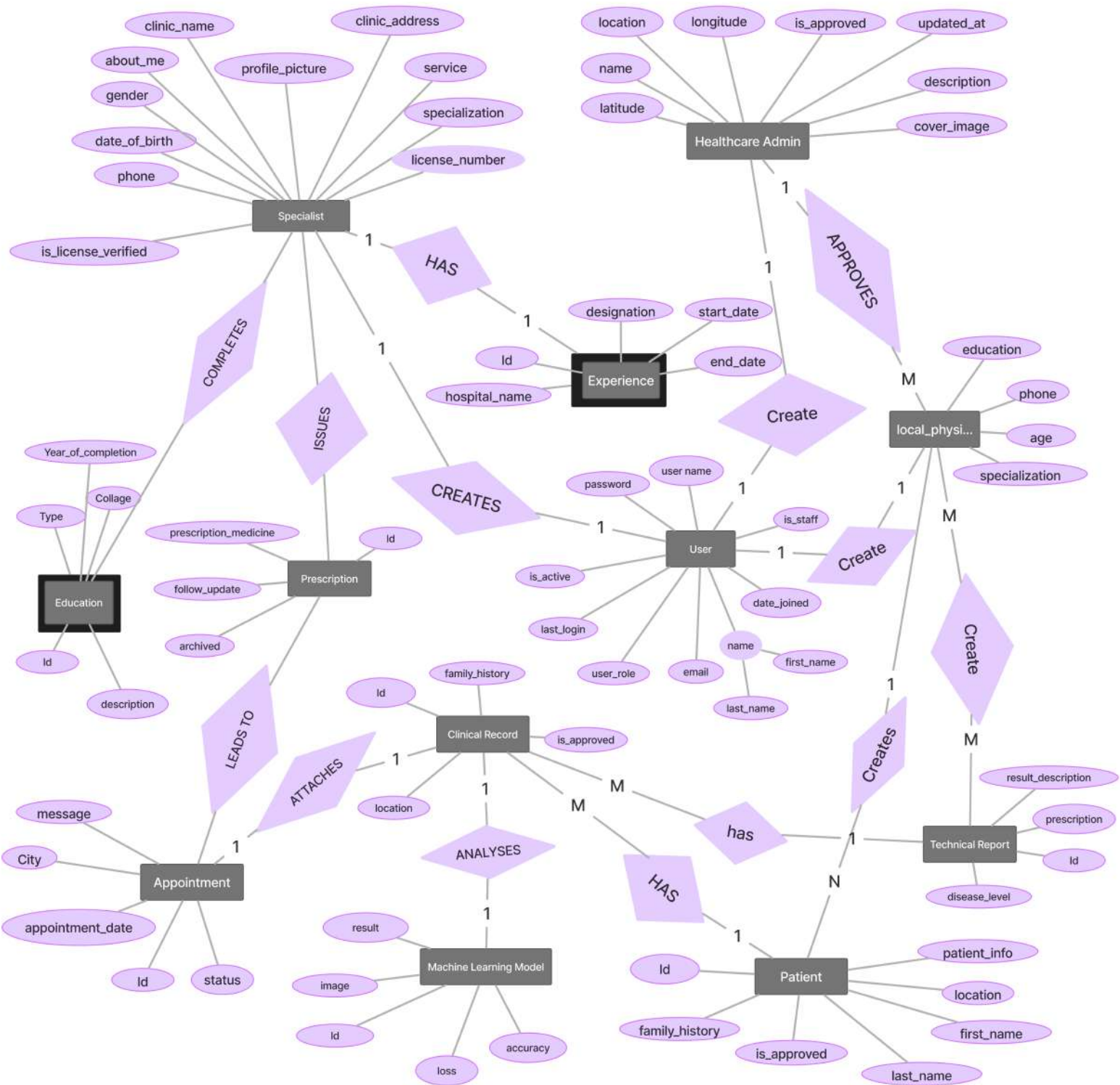


Figure 3.8: ER Diagram

#### **3.2.2.5 API Design (Restful API in Django, Djoser):**

Django REST Framework (DRF) offers a standardized architecture for building scalable web services. It provides powerful tools for serialization, authentication, and permissions, making it easier to develop and maintain robust APIs.

#### **3.2.2.6 Model Integration(ML Integration with Back-end)**

Integrating machine learning models, such as ResNet50, into a Django backend necessitates meticulous orchestration of various technologies and libraries. Our project adopted TensorFlow to facilitate this integration but initially encountered compatibility challenges due to version discrepancies. Through rigorous review of relevant documentation and diligent troubleshooting efforts, we successfully resolved these issues. The implemented solution ensured compatibility between the Django backend, TensorFlow framework, and the pre-trained ResNet50 model, thereby enabling seamless communication and integration. This experience highlights the crucial role of comprehensive research, meticulous attention to detail, and adaptability when integrating complex technological components to achieve project objectives.

```

import os
from tensorflow.keras.models import load_model
from tensorflow.keras.layers import BatchNormalization
from django.conf import settings
import numpy as np
import cv2

def preprocess_image_for_eye(image):
    # Resize the image to match the input size required by the model
    image_size = 224
    new_image = cv2.imdecode(np.fromstring(image.read(), np.uint8), cv2.IMREAD_COLOR)
    new_image = cv2.resize(new_image, (image_size, image_size))
    new_image = new_image.astype(np.float32)
    return new_image

def predict_with_eye_model(image):

    # access and load the model
    model_path = os.path.join(settings.MEDIA_ROOT, 'models', 'eye_model.h5')
    model = load_model(model_path)

    # access the preprocessed model
    processed_image = preprocess_image_for_eye(image)

    # perform inference
    predictions = model.predict(np.expand_dims(processed_image, axis=0))
    predicted_classes = np.argsort(predictions[0])[::-1][:3] # Get indices of top 3
classes

    class_labels = ["Normal", "Cataract", "Diabetes", "Glaucoma", "Hypertension",
"Age Issues", "Other"]

    top_predictions = []
    for class_index in predicted_classes:
        class_label = class_labels[class_index]
        probability = predictions[0][class_index]
        top_predictions.append((class_label, probability))

```

Figure 3.9: Code Snippet

# Ethio Health Hub <sup>v1</sup>

[ Base URL: 127.0.0.1:8000/ ]  
<http://127.0.0.1:8000/swagger/?format=openapi>

[Contact the developer](#)

Schemes

HTTP

[Django Login](#)
[Authorize](#)

## appointment

- GET /appointment/ appointment\_list
- POST /appointment/ appointment\_create
- GET /appointment/detail/{id} appointment\_detail\_read
- GET /appointment/{id}/ appointment\_read

- PUT /appointment/{id}/ appointment\_update
- PATCH /appointment/{id}/ appointment\_partial\_update
- DELETE /appointment/{id}/ appointment\_delete

## appointments

- GET /appointments/specialist/{specialist\_id} appointments\_specialist\_read
- GET /appointments/station/{station\_id} appointments\_station\_read

## auth

- POST /auth/jwt/create/ auth\_jwt\_create\_create
- POST /auth/jwt/refresh/ auth\_jwt\_refresh\_create

Figure 3.10: Some APIs used

### 3.2.3 Machine Learning Development

#### 3.2.3.1 Data Acquisition

The cornerstone of any successful machine learning project lies in its data. This is especially true in the realm of medical diagnosis, where accurate and diverse data is paramount for training effective models. This project delves into the world of data acquisition for two crucial areas: skin cancer and eye disease detection.

**Skin Cancer: the HAM10000 Dataset** Early and precise diagnosis of skin cancer is vital for successful treatment. Traditionally, dermatologists rely on visual examination, sometimes supplemented by specialized techniques. However, advancements in technology pave the way for automated methods to assist in skin cancer detection.

The HAM10000 dataset serves as a valuable resource for this endeavor. This publicly available collection boasts over 10,000 dermoscopic images, capturing a wide spectrum of skin conditions. The dataset allows researchers to train and evaluate machine learning models against the expertise of human dermatologists in identifying various types of skin cancer. The rich variety of data within HAM10000 is key. It encompasses a diverse range of lesions, including the following classifications:

1. Melanocytic nevi: these are common moles that are usually benign.
2. Melanoma: the most serious form of skin cancer, characterized by uncontrolled growth of pigment-producing cells.
3. Benign keratosis-like lesions: noncancerous growths that resemble keratosis, a rough, scaly patch on the skin.
4. Basal cell carcinoma: the most common type of skin cancer, typically slow-growing and highly treatable when detected early.
5. Actinic keratoses: precancerous lesions caused by sun exposure.
6. Hemangiomas: abnormal blood vessel growths on the skin.
7. Dermatofibroma: a noncancerous nodule in the deeper layer of the skin.

This comprehensive classification scheme within HAM10000 allows the model to learn the distinct visual characteristics of each skin condition. By differentiating these subtle variations, the model can potentially aid in more accurate skin cancer detection.

Observing the age range of the data gathered:

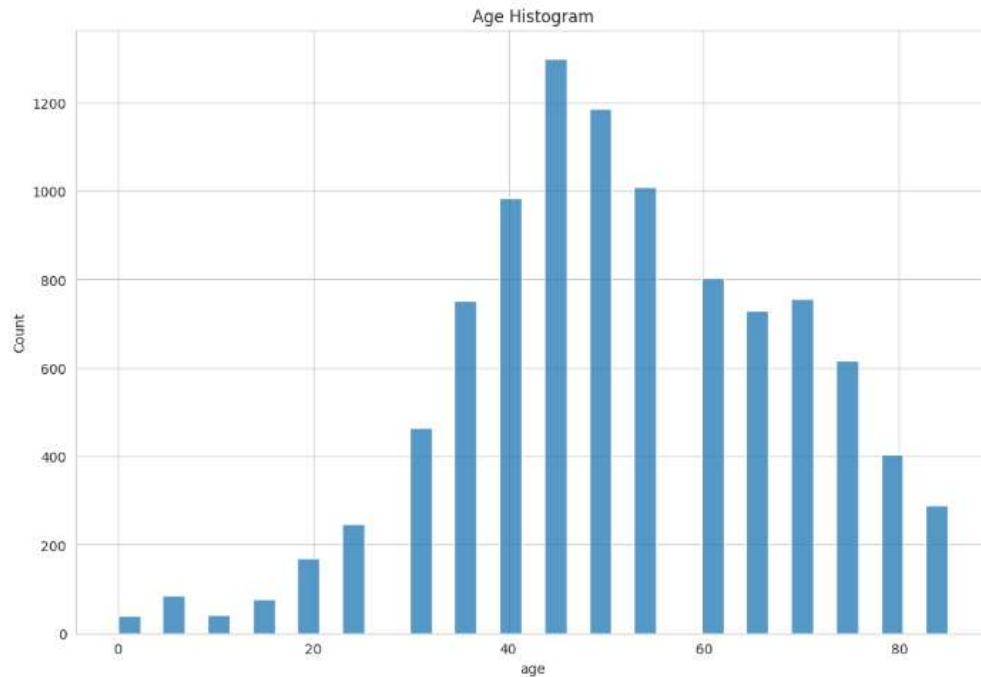


Figure 3.11: HAM10000 Dataset based on age

Furthermore, analyzing the body parts that have higher vulnerability for skin cancer the following graph summarize it as:



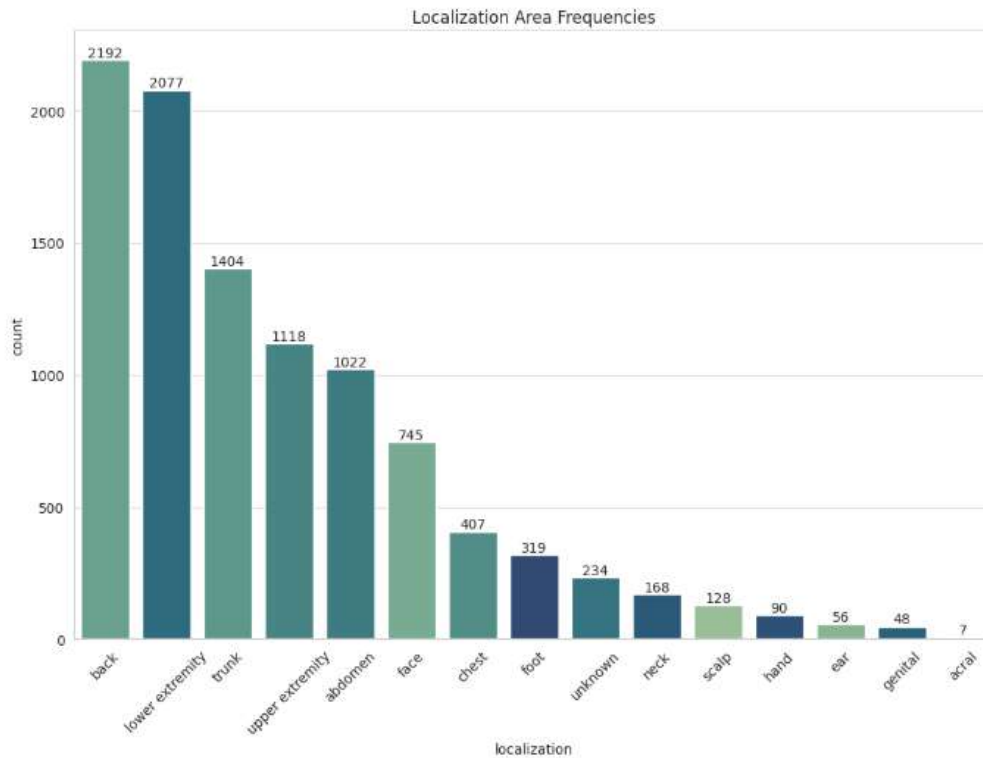


Figure 3.12: Skin Cancer Vulnerable Body Parts

The following graph reveals a gender bias in skin cancer prevalence, with a higher frequency observed in males compared to females. Furthermore, the data suggests that the melanocytic nevi type of skin cell exhibits a significantly greater susceptibility to skin cancer compared to other skin cell types.

Observation of skin cancer cases indicates that the 'Melanocytic nevi' type of skin cell is the most prevalent, accounting for a significantly higher proportion of instances compared to other cell types.

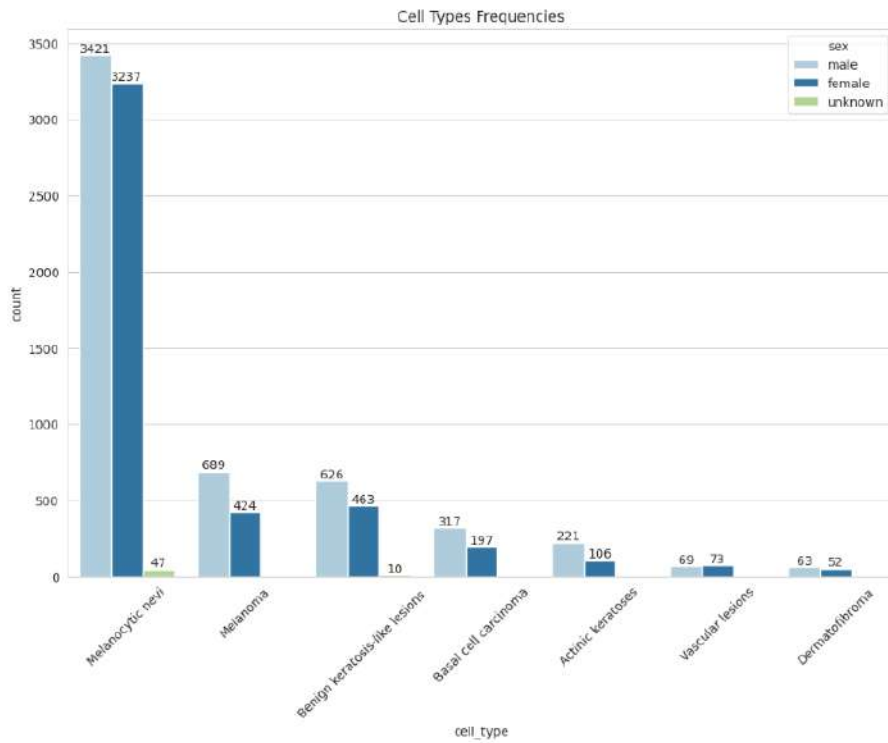


Figure 3.13: Gender Bias for different skin cancer types

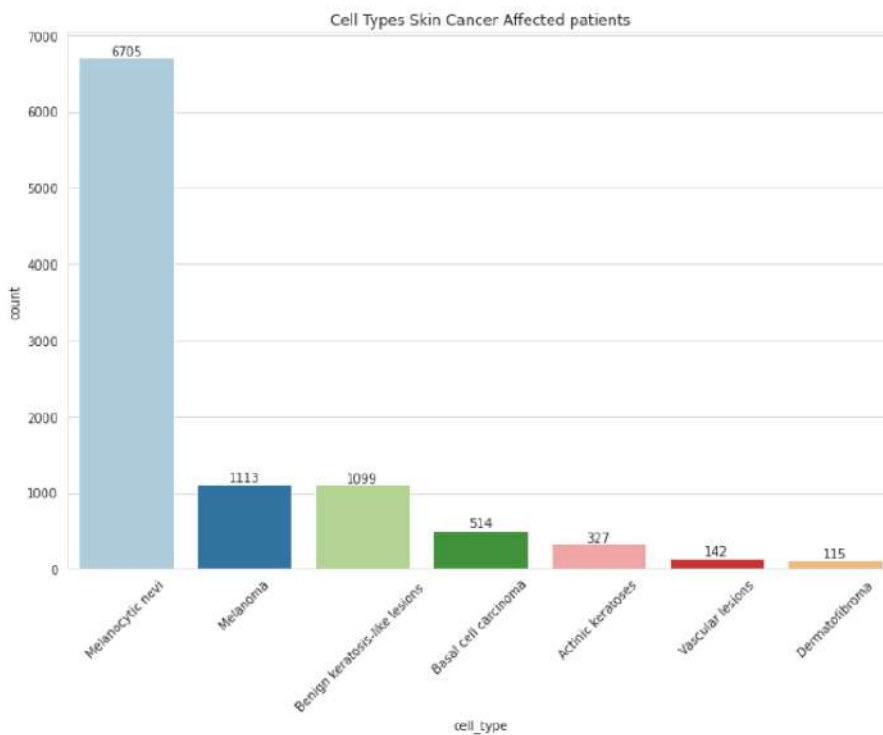


Figure 3.14: Cell Type Frequency

**Eye Disease (The ODIR Dataset):** the potential of automated image analysis extends beyond skin cancer. The Ocular Disease Intelligent Recognition (ODIR) dataset showcases another exciting application in eye disease detection.

ODIR offers a unique glimpse into real-world medical scenarios. It comprises data from over 5,000 patients, including color fundus photographs of both eyes alongside diagnoses provided by doctors. This data provides valuable insights as it reflects a practical setting, with images captured by various cameras in different hospitals. The quality of the data is further ensured by meticulous annotations by trained professionals.

The ODIR dataset categorizes patients into eight distinct groups based on their eye condition, including:

1. Normal(N): healthy eyes with no signs of disease.
2. Diabetes (D): diabetic retinopathy, a complication of diabetes that can damage the blood vessels in the retina.
3. Glaucoma (G): a group of eye diseases that damage the optic nerve, leading to vision loss.
4. Cataract (C): clouding of the lens of the eye, which can cause blurry vision.
5. Age-related Macular Degeneration (A): a deterioration of the macula, the central part of the retina responsible for sharp central vision.
6. Hypertension (H): high blood pressure, which can contribute to eye problems like diabetic retinopathy.
7. Pathological Myopia (M): a severe form of nearsightedness that can increase the risk of retinal detachment.
8. Other diseases/abnormalities (O): this category encompasses various less common eye conditions.

This detailed classification scheme within ODIR allows for the development of models that can identify a broad spectrum of eye diseases. By analyzing fundus photographs, these models could potentially assist doctors in earlier diagnoses and personalized treatment plans for patients.

Visualizing the data insights, based on age, the prevalence of eye disease increases as age increase. Based on gender male shows great susceptibility than female in general.

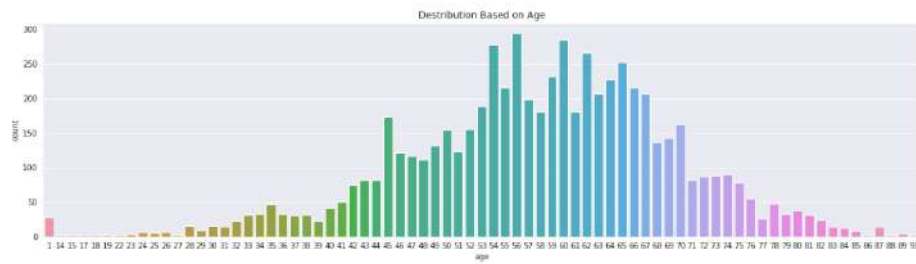


Figure 3.15: Distribution based on Age

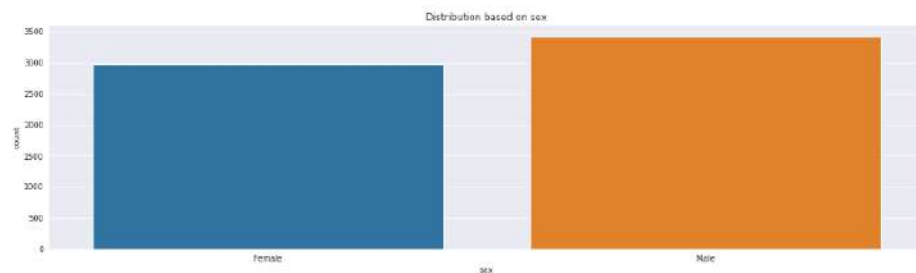


Figure 3.16: Distribution based on Gender

By exploring these two datasets, HAM10000 and ODIR, we gain a deeper understanding of the importance of data acquisition in developing robust machine-learning models for medical diagnosis.

The quality, diversity, and real-world relevance of these datasets are crucial for creating models that can effectively assist healthcare professionals in the fight against various diseases.

### 3.2.3.2 Data Preprocessing

Starting with the Skin Cancer Image dataset the pre-processing steps will be discussed further.

**Data Cleaning:** the process started by analyzing the number of null values in the dataset.

```
In [6]: skin_df.isnull().sum()
```

```
Out[6]:
lesion_id      0
image_id       0
dx             0
dx_type        0
age           57
sex            0
localization   0
path           0
cell_type      0
cell_type_idx  0
dtype: int64
```

Figure 3.17: Code Snippet checking for Null values

After filling the null values with the mean value. The following code snippet performs several image processing tasks on a DataFrame named `skin_df` containing a column named `path` with image file paths. Let's break down each step:

```
# from tensorflow.keras.preprocessing.image import load_img, img_to_array
image_size = 224
skin_df['image'] = None
for idx, row in skin_df.iterrows():
    try:
        image = cv2.imread(row['path'], cv2.IMREAD_COLOR)
        image = cv2.resize(image, (image_size, image_size))
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        skin_df.at[idx, 'image'] = np.array(image)
    except Exception as e:
        print(e)
        continue

skin_df.head()
```

Figure 3.18: Code Snippet checking for Null values

### 1. Define Image Size:

- (a) The variable `image_size` is set to 224. This defines the target width and height for resizing the images.

### 2. Load Images:

- (a) The code iterates through each row of the data frame using `iterrows()`.
- (b) Inside the loop, it tries to read an image using OpenCV's `cv2.imread` function.
- (c) The function takes two arguments:
  - i. `row['path']`: This retrieves the image file path from the current row's path column.
  - ii. `cv2.IMREAD_COLOR`: This flag specifies that the image should be loaded in color mode (BGR by default in OpenCV).

### 3. Error Handling:

- (a) The code uses a try-except block to handle potential errors during image reading.
- (b) If an exception (Exception) occurs while reading an image (e.g., file not found, corrupted image), the error message is printed using `print(e)`.
- (c) The `continue` statement skips to the next iteration, avoiding processing errors further.

### 4. Resize Images:

- (a) If the image is loaded successfully, it's resized using `cv2.resize`.
- (b) This function takes two arguments:
  - i. The loaded image (`image`)
  - ii. A tuple specifying the new width and height (`(image_size, image_size)`)

### 5. Convert Color Space:

- (a) The resized image is converted from BGR (OpenCV's default color format) to RGB using `cv2.cvtColor`.
- (b) This conversion is important because many machine-learning libraries and tools expect images in RGB format.

### 6. Update DataFrame:

- (a) The processed image (resized and converted to RGB) is converted to a NumPy array using `np.array`.
- (b) The array is then assigned back to the image column of the DataFrame at the current index (`idx`) using `skin_df.at[idx, 'image']`.

## Eye Disease: The ODIR Dataset Preprocessing

In eye disease The CSV file is a little bit hard to handle so we processed the image using a function called `process_dataset` which will create a new label and store the data in the correct ids.

This section describes the process of pre-processing image data for use in a machine learning model built with TensorFlow Keras. The pre-processing steps ensure the images are in a consistent format that the model can efficiently process.

### Function: `dataset_generator`

The `dataset_generator` function takes a list of image filenames (`imagecategory`) belonging to a specific class and their corresponding class label (`label`) as input. It then performs the following steps:

1. Define Image Size: The function uses a predefined `image_size` variable to determine the target width and height for resizing the images.
2. Image Path Construction: For each image filename in `imagecategory`, the function constructs the complete file path by combining the base directory (`dataset_dir`) where pre-processed images are stored and the filename.
3. Image Loading and Resizing:
  - (a) The function attempts to read the image from the constructed path using OpenCV's `cv2.imread` function in color mode (`cv2.IMREAD_COLOR`).
  - (b) If successful, it resizes the image to the `image_size` using `cv2.resize`.
  - (c) In case of errors (e.g., corrupted image, file not found), the function gracefully skips the problematic image and continues processing the next one.
4. Data Creation:
  - (a) For each successfully loaded image, it's converted into a NumPy array using `np.array`.
  - (b) A list containing the preprocessed image array and its corresponding class label (`[np.array(image), np.array(label)]`) is created.
  - (c) This list is appended to a larger dataset list that accumulates all preprocessed data.



## 5. Data Shuffling:

- (a) After processing all images in the category, the function shuffles the dataset list using `random.shuffle`. Shuffling randomizes the order of data samples, which is often beneficial for training machine learning models.

## 6. Return Value:

- (a) The function returns the final dataset list containing preprocessed image data (as NumPy arrays) along with their corresponding class labels. This dataset can be directly used for training a TensorFlow Keras model.

```
# preprocess the image and store the image in preprocessed_images directory
from tensorflow.keras.preprocessing.image import load_img, img_to_array
dataset_dir = "preprocessed_images"
image_size=224
labels = []
dataset = []
def dataset_generator(dataset,imagecategory , label):
    for img in tqdm(imagecategory):
        imgpath = os.path.join(dataset_dir,img);

        #now we try to read the image and resize it accordingly
        try:
            image = cv2.imread(imgpath,cv2.IMREAD_COLOR)
            image = cv2.resize(image,(image_size,image_size))
        except:
            continue;
        dataset.append([np.array(image),np.array(label)]);
    random.shuffle(dataset);

    return dataset;
```

Figure 3.19: Preprocessing code snippet

### 3.2.3.3 Model Building with Resnet-50

#### Studies on Machine Learning Model Selection

**VGG(Visual Geometry Group):** is a deep Convolutional Neural Network (CNN) architecture with multiple layers, specifically designed for image recognition tasks. The "deep" in VGGNet refers to the number of layers, with VGG-16 and VGG-19 comprising 16 and 19 convolutional layers, respectively. The VGG19 model includes 19 layers: 16 convolutional layers, 3 fully connected layers, 5 max pooling layers, and a softmax layer. The input layer takes a fixed-size RGB image of 224x224 pixels. The convolutional layers utilize 3x3 filters

with ReLU activation, and max pooling layers are used to reduce spatial dimensions and the number of parameters. The three fully connected layers have 4096 neurons each, with the final layer having 1000 neurons for classification. The output layer is a softmax function that provides a probability distribution over 1000 classes.

**DenseNet(Densely Connected Convolutional Network):** is a deep Convolutional Neural Network (CNN) architecture designed to ensure maximum information flow between layers. Unlike traditional CNNs where each layer has its own set of parameters, DenseNet connects each layer to every other layer in a feed-forward fashion, allowing the layers to reuse features and mitigate the vanishing gradient problem. The architecture is divided into dense blocks, where each layer receives input from all preceding layers and passes its own feature maps to all subsequent layers. This connectivity pattern results in a significant reduction in the number of parameters, improves parameter efficiency, and encourages feature reuse. DenseNet models are typically denoted as DenseNet-121, DenseNet-169, DenseNet-201, and DenseNet-264, where the numbers indicate the total number of layers in the network. Each DenseNet consists of convolutional layers, batch normalization, ReLU activation, and transition layers that include 1x1 convolutions and 2x2 average pooling. DenseNet has been shown to achieve high accuracy with fewer parameters and is highly effective on various image recognition tasks, including those in the ImageNet and CIFAR datasets.

**ResNet(Residual Neural Network):** is a deep learning architecture that aims to address the problem of vanishing gradients and facilitate the training of very deep neural networks. ResNet is the introduction of residual connections or skip connections that allow the information to bypass certain layers in the network. These skip connections enable the network to learn residual mappings, which are the differences between the desired output and the current output of a layer. By learning these residuals, the network can focus on refining the learned features rather than starting from scratch at each layer. ResNet-50 consists of 50 layers, including convolutional layers, residual blocks, and fully connected layers. The general architecture of ResNet-50:

1. Input Layer: Accepts the input image or data.
2. Convolutional and Pooling Layers: The initial layers perform convolution operations

on the input to extract features and reduce spatial dimensions. Pooling layers, typically using max pooling, further downsample the feature maps.

3. **Residual Blocks:** The core building blocks of ResNet, these blocks contain stacked convolutional layers with skip connections. Specifically, ResNet-50 uses bottleneck blocks, which consist of three convolutional layers:  $1 \times 1$ ,  $3 \times 3$ , and  $1 \times 1$ . Each bottleneck block has shortcut connections that bypass some convolutional layers, allowing the network to learn residual mappings. ResNet-50 has several residual blocks with different numbers of bottleneck blocks.
4. **Global Average Pooling:** After the residual blocks, global average pooling is applied to reduce the spatial dimensions of the feature maps to a fixed size.
5. **Fully Connected Layers:** The pooled features are fed into fully connected layers for classification or other specific tasks. These layers map the learned features to the desired number of output classes.
6. **Softmax Activation:** The final layer applies the softmax activation function to produce probability scores for each output class.

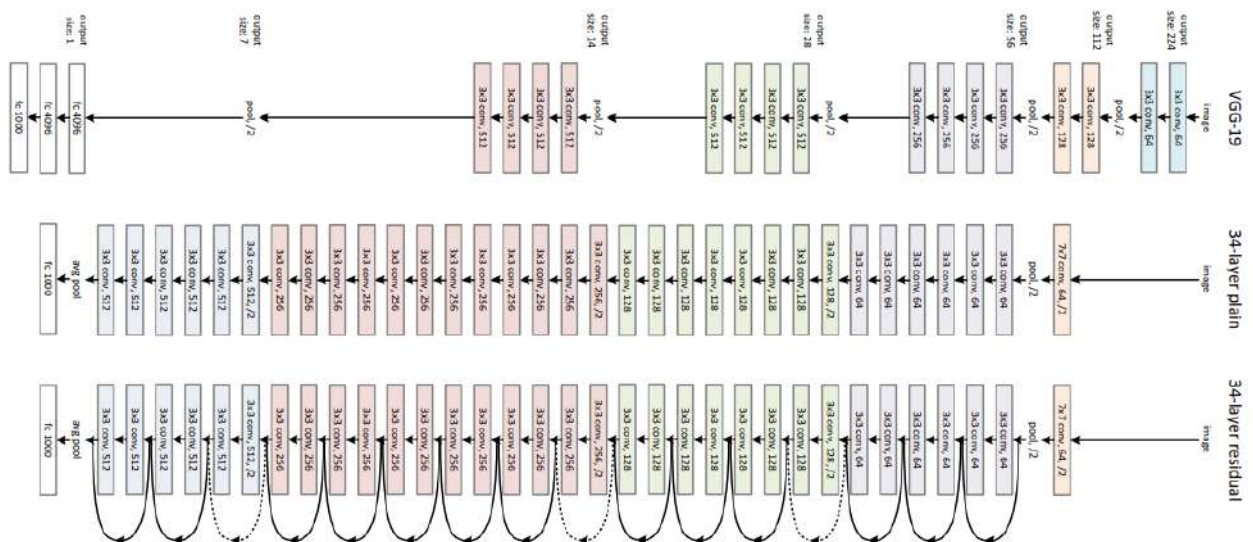


Figure 3.20: Architecture of ResNet-50

Throughout the project, ResNet is utilized as the primary machine learning model. Its innovative approach effectively addresses the vanishing gradient problem, enabling the training of significantly deeper networks compared to its predecessors.

While DenseNet offers parameter efficiency through dense connectivity, it also entails heightened computational demands. ResNet strikes a delicate balance between model complexity and computational efficiency, making it an ideal choice for the project's requirements.

### **Building a Deep Learning Model for Eye Disease Classification Using ResNet-50**

Our machine learning model leverages the powerful ResNet-50 architecture as its backbone for feature extraction, followed by additional layers designed to classify various eye diseases accurately. The model begins with an input layer that standardizes the input images to dimensions of 224x224x3 (height, width, color channels), ensuring compatibility with the ResNet-50 model. The pre-trained ResNet-50 model, which has demonstrated exceptional performance on the ImageNet dataset, serves as the primary feature extractor. It processes the input images through its 50 convolutional layers, equipped with skip connections, to effectively mitigate the vanishing gradient problem and extract high-level features. The output from ResNet-50 is a 3D tensor of shape (7, 7, 2048), which is then flattened into a 1D tensor of shape (100352) by a Flatten layer. This transformation is crucial as it prepares the extracted features for the subsequent fully connected layers.

Following the Flatten layer, a Dense layer with 256 units is employed to learn complex patterns and representations from the flattened features. This layer acts as a critical component in translating the high-dimensional feature space into meaningful patterns that can aid in classification. To enhance the model's training efficiency and stability, we incorporate a Batch Normalization layer, which normalizes the output of the Dense layer by adjusting and scaling the activations. This step reduces the internal covariate shift, thereby stabilizing and accelerating the training process.

The final layer in the model is another Dense layer with 7 units, each corresponding to one of the target classes: Normal, Glaucoma, Cataract, Age-related Macular Degeneration, Hypertension, Pathological Myopia, and Other diseases/abnormalities. This layer employs a softmax activation function to produce a probability distribution over the classes, ensuring that the output probabilities sum to one, with the highest probability indicating the predicted class. This structured approach, combining ResNet-50's deep feature extraction capabilities with additional fully connected and normalization layers, provides a robust framework for accurately classifying a diverse set of eye diseases based on medical images.

## Model Hyperparameters

1. Number of epochs: 70
2. Batch size: 32
3. Optimizer: 'adam'
4. Loss function: 'categorical\_crossentropy'
5. Evaluation metric: 'accuracy'

Our machine learning model leverages the powerful ResNet-50 architecture as its backbone for feature extraction, followed by additional layers designed to classify various eye diseases accurately. The model begins with an input layer that standardizes the input images to dimensions of 224x224x3 (height, width, color channels), ensuring compatibility with the ResNet-50 model. The pre-trained ResNet-50 model, which has demonstrated exceptional performance on the ImageNet dataset, serves as the primary feature extractor. It processes the input images through its 50 convolutional layers, equipped with skip connections, to effectively mitigate the vanishing gradient problem and extract high-level features. The output from ResNet-50 is a 3D tensor of shape (7, 7, 2048), which is then flattened into a 1D tensor of shape (100352) by a Flatten layer. This transformation is crucial as it prepares the extracted features for the subsequent fully connected layers.

Following the Flatten layer, a Dense layer with 256 units is employed to learn complex patterns and representations from the flattened features. This layer acts as a critical component in translating the high-dimensional feature space into meaningful patterns that can aid in classification. To enhance the model's training efficiency and stability, we incorporate a Batch Normalization layer, which normalizes the output of the Dense layer by adjusting and scaling the activations. This step reduces the internal covariate shift, thereby stabilizing and accelerating the training process.

The final layer in the model is another Dense layer with 7 units, each corresponding to one of the target classes: Normal, Glaucoma, Cataract, Age-related Macular Degeneration, Hypertension, Pathological Myopia, and Other diseases/abnormalities. This layer employs a softmax activation function to produce a probability distribution over the classes, ensuring that the output probabilities sum to one, with the highest probability indicating the predicted class. This structured approach, combining ResNet-50's deep feature extraction capabilities

with additional fully connected and normalization layers, provides a robust framework for accurately classifying a diverse set of eye diseases based on medical images.

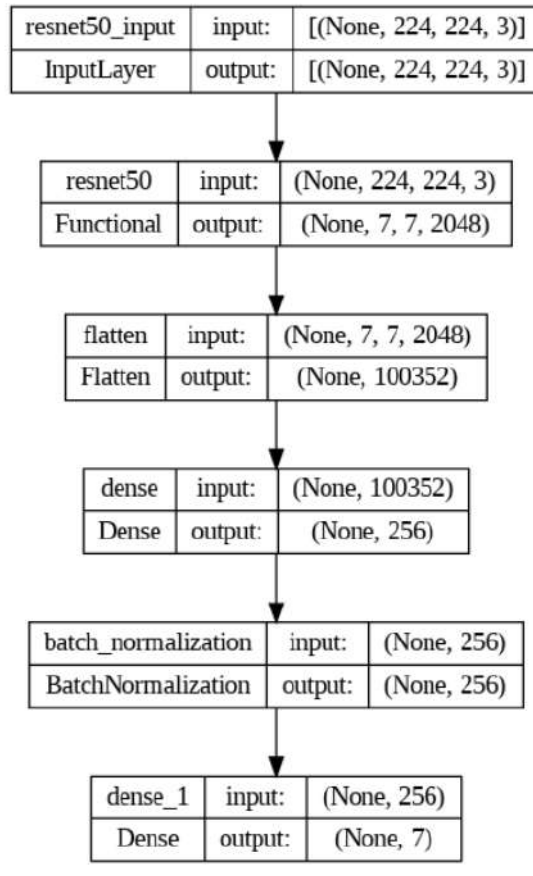


Figure 3.21: Layers of Trained Model for Eye Disease

```

94765736/94765736 [=====] - 4s 0us/step
Model: "sequential"

```

| Layer (type)                              | Output Shape       | Param #  |
|-------------------------------------------|--------------------|----------|
| resnet50 (Functional)                     | (None, 7, 7, 2048) | 23587712 |
| flatten (Flatten)                         | (None, 100352)     | 0        |
| dense (Dense)                             | (None, 256)        | 25690368 |
| batch_normalization (Batch Normalization) | (None, 256)        | 1024     |
| dense_1 (Dense)                           | (None, 7)          | 1799     |

```

=====
Total params: 49280903 (187.99 MB)
Trainable params: 25692679 (98.01 MB)
Non-trainable params: 23588224 (89.98 MB)

```

Figure 3.22: Code Snippet showing Trained Layers

## **Building a Deep Learning Model for Skin Disease Classification Using ResNet-50**

For skin model, we developed a machine learning model using the ResNet-50 architecture to classify various skin diseases, including Melanocytic nevi, Melanoma, Benign keratosis-like lesions, Basal cell carcinoma, Actinic keratoses, Vascular Lesions, and Dermatofibroma. The model architecture starts with an input layer designed to accept images of size 224x224 with three color channels (RGB), ensuring standardized input dimensions compatible with ResNet-50. The pre-trained ResNet-50 model, with its 50 convolutional layers and residual connections, is utilized for feature extraction. This model, pre-trained on the ImageNet dataset, outputs a 3D tensor of shape (7, 7, 2048), effectively capturing high-level features from the input images. Next, a Flatten layer converts this 3D tensor into a 1D tensor with 100352 elements, preparing the extracted features for the subsequent fully connected layers.

Following the flattening, a Dense layer with 256 units learns complex patterns and representations from the flattened features. To stabilize and accelerate the training process, a Batch Normalization layer normalizes the activations from the Dense layer, reducing internal covariate shift. A Dropout layer with a certain probability is then added to prevent overfitting by randomly dropping units during training. Another Dense layer with 128 units further reduces the dimensionality and refines the feature representation. This is followed by another Batch Normalization layer to ensure consistent and efficient training. Finally, the output Dense layer, with seven units corresponding to the seven target classes, produces the final classification output using a softmax activation function. This structured approach leverages the powerful feature extraction capabilities of ResNet-50, along with additional fully connected, batch normalization, and dropout layers, to create a robust model capable of accurately classifying a variety of skin diseases.

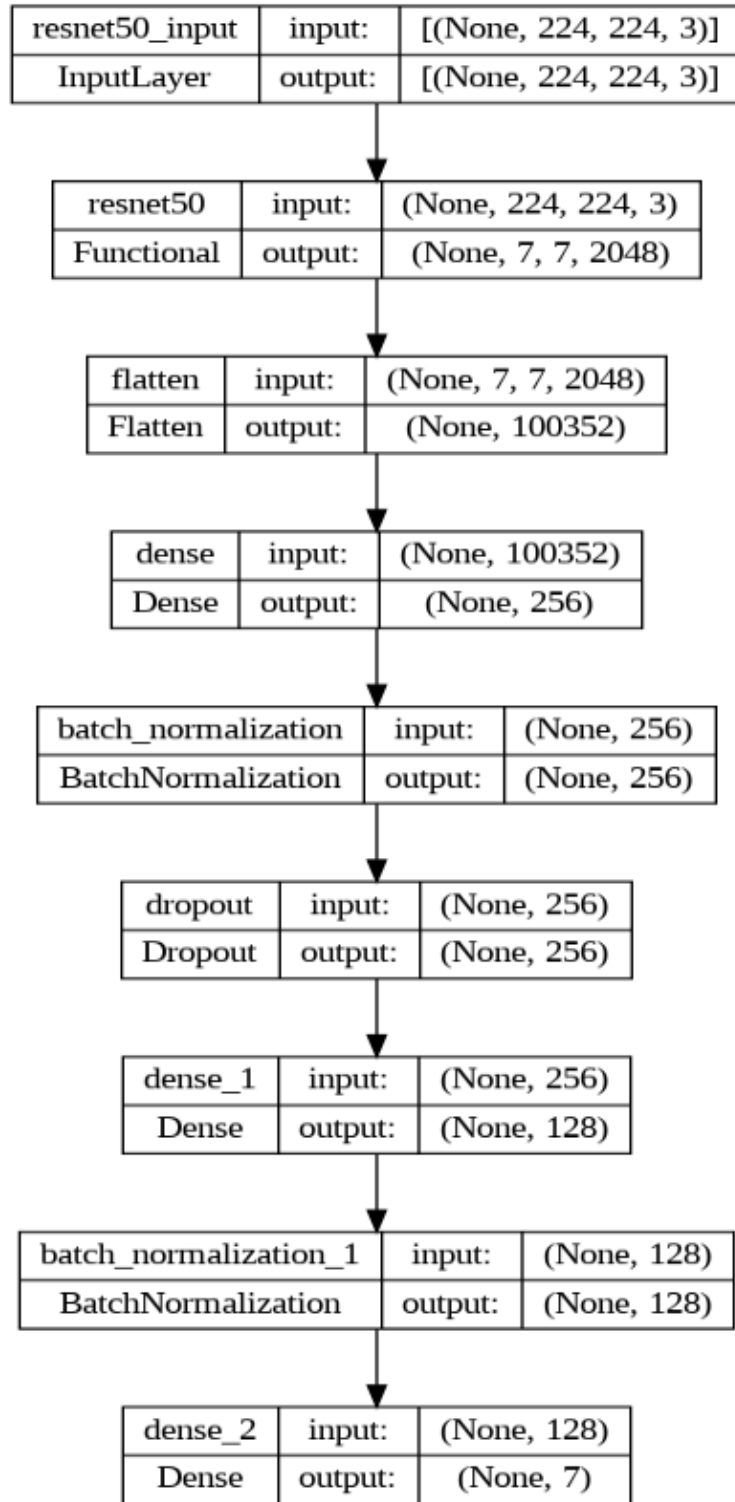


Figure 3.23: Layers of Trained Model for Skin Disease



### 3.2.3.4 Model Testing and Evaluation

#### Skin Model Evaluation

In the Model Evaluation step, we meticulously assess the performance of our trained model. This involves checking the testing accuracy and validation accuracy to gauge how well the model generalizes to unseen data. Additionally, we plot a confusion matrix to visualize the model's performance in classifying different classes, providing insights into where the model may be struggling. Furthermore, we analyze the count of misclassified images for each class, helping to identify which classes are more challenging for the model. Overall, this step is crucial for understanding the strengths and weaknesses of the model and for making informed decisions on how to improve its performance.

```
loss, accuracy = model.evaluate(x_test, y_test, verbose=1)
loss_v, accuracy_v = model.evaluate(x_validate, y_validate, verbose=1)
print("Validation: accuracy = %f ; loss_v = %f" % (accuracy_v, loss_v))
print("Test: accuracy = %f ; loss = %f" % (accuracy, loss))
model.save("model.h5")
```

Figure 3.24: Code Snippet used to Evaluate the Model

To enhance model performance, we employed key strategies. Data preprocessing ensured data suitability, including scaling and normalization. Data augmentation diversified the training dataset. Hyperparameters like learning rate and batch size were fine-tuned, and regularization techniques like dropout were applied. Model architecture was optimized, and training was monitored using a separate validation set to prevent overfitting. These strategies collectively improved model performance by reducing loss and increasing accuracy and finally we got this results.

```
2003/2003 [=====] - 1s 694us/step
802/802 [=====] - 0s 527us/step
Validation: accuracy = 0.785536 ; loss_v = 0.586728
Test: accuracy = 0.764853 ; loss = 0.616134
```

Figure 3.25: Out put for the evaluation

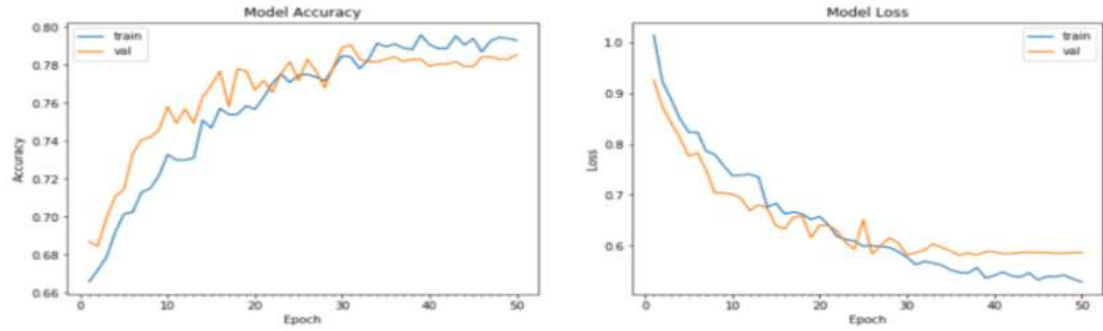


Figure 3.26: Model Accuracy and Model Loss representation in graph

Additionally, we analyzed the model's performance using a confusion matrix, which visually represented the model's ability to correctly classify or predict each class. The confusion matrix graph provided insights into where the model was performing well and where it was struggling, helping us further refine the model for better accuracy.

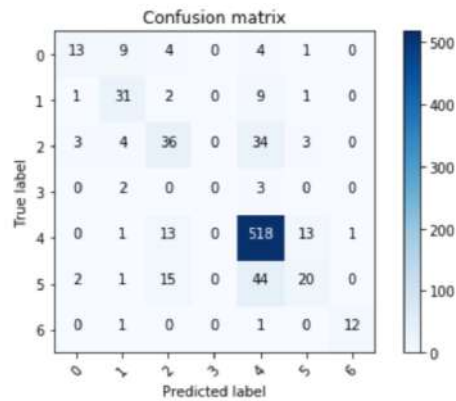


Figure 3.27: Confusion Matrix

## Eye Model Evaluation

In the accuracy graph, both training and validation accuracy increase rapidly and then plateau, reflecting the model's ability to quickly learn and recognize patterns in the data. The training accuracy remains high and consistent, while the validation accuracy, despite minor fluctuations, closely follows the training accuracy. This trend indicates that the model maintains good generalization capabilities. The final evaluation of the model on the test set reveals a loss of 0.2190 and an impressive accuracy of 93.20%. These metrics suggest that the model's predictions are close to the true labels and that it performs exceptionally well in classifying eye diseases.

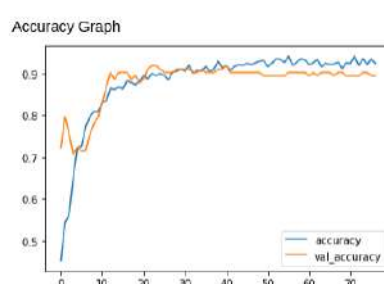


Figure 3.28: Model Accuracy

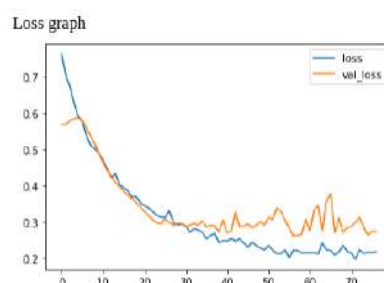


Figure 3.29: Model Loss

## 3.2.4 Integration and Deployment

### 3.2.4.1 Database and Backend Integration

1. Database Selection: the initial development phase of this project utilized SQLite as the primary database management system (DBMS). This selection was motivated by SQLite's inherent simplicity and straightforward setup process, which significantly accelerated the prototyping and testing stages. However, for production deployment, a transition to PostgreSQL was deemed necessary. PostgreSQL's robust design, scala-

bility, and demonstrably adept handling of large-scale applications with complex data needs made it the optimal choice. Furthermore, PostgreSQL's comprehensive feature set, including support for JSON data types, full-text search capabilities, and concurrent transactions, perfectly aligned with the application's anticipated database requirements within a production environment.

2. **Integration Process:** due to Django's inherent support for PostgreSQL, the integration between the Django backend and the PostgreSQL database management system (DBMS) proved to be straightforward. We effectively utilized Django's object-relational mapper (ORM) to define application models that directly mapped to corresponding PostgreSQL tables. This approach fostered seamless interaction between the backend logic and the database, enabling efficient data storage and retrieval operations. Furthermore, Django's ORM inherently enforces data integrity and consistency constraints, safeguarding the application's data reliability.

#### **3.2.4.2 Back-end and Front-end Integration:**

1. **Library Usage:** to facilitate communication between the backend and frontend layers, our application employed libraries such as Axios and Fetch for managing HTTP requests associated with CRUD (Create, Read, Update, Delete) operations. Axios provided a user-friendly and intuitive application programming interface (API) for making asynchronous requests from the frontend to the backend servers. Conversely, Fetch offered a native approach for fetching resources across the network, leveraging built-in browser functionalities. This combination of libraries ensured flexibility and compatibility in data exchange between the frontend and backend components.
2. **Integration Approach:** the integration between the backend and frontend was achieved through the implementation of RESTful API endpoints within the Django framework, facilitated by Django REST Framework. These endpoints functioned as the well-defined communication channels between the two tiers, enabling the frontend to seamlessly interact with the backend services. Adhering to a modular approach, each API endpoint was meticulously designed to correspond to a specific functionality or resource within the application, such as user authentication, data retrieval, or data manipulation. This modular structure promotes code organization, maintainability, and scalability.

### **3.2.5 Testing and Deployment Documentation**

The Ethio-Virtual Healthcare web application platform adheres to a rigorous multi-phased testing strategy designed to guarantee its reliability, and overall functionality.

This strategy encompasses various testing methodologies to ensure comprehensive coverage. In the initial phase, unit testing focuses on validating the functionality of individual software components in isolation. This is followed by integration testing, which verifies the seamless interaction and data exchange between these components.

By achieving a high degree of test coverage, exceeding 80% in this instance. Ethio-Virtual Healthcare web application platform can be successfully deployed, offering a secure and reliable service.

In conclusion, this chapter has comprehensively explored the methodological approaches employed to establish a thorough understanding of the subject matter, followed by the design and implementation of the system. The subsequent chapter will delve into the insights uncovered and the outcomes achieved through this project.

# Chapter 4

## RESULT and DISCUSSIONS

The following chapter provides details of result found in the previous chapter with further analysis and discussion.

### 4.1 Result

The integration of Artificial Intelligence (AI) and Machine Learning (ML) algorithms in the diagnostic process has significantly enhanced the precision and timeliness of diagnosing eye and skin diseases. These advanced technologies enable the identification of subtle patterns and anomalies in medical images that might be overlooked by traditional diagnostic methods. Consequently, there has been a noticeable reduction in diagnostic errors, which traditionally arise from human limitations and variability. AI and ML algorithms provide consistent and reproducible results, ensuring a higher standard of diagnostic accuracy across various clinical settings. Early and precise diagnosis facilitated by these technologies can lead to better patient outcomes through timely intervention and treatment.

The establishment of Virtual Healthcare Centers represents a significant advancement in the accessibility of specialized medical care. These centers leverage telemedicine platforms to offer remote consultations, thereby bridging the gap for patients in underserved areas who lack easy access to specialized eye and skin care services. The virtual model of healthcare delivery significantly decreases the need for patients to travel long distances to consult with specialists, leading to substantial savings in both time and costs. This model not only benefits patients by providing timely and convenient access to expert opinions but also helps to alleviate the burden on urban healthcare facilities by distributing the demand more evenly across the healthcare system.

The deployment of AI and ML in healthcare extends beyond diagnostics to the automation of various routine tasks, thereby enhancing overall healthcare efficiency. Automation helps in streamlining administrative tasks such as appointment scheduling, patient triage, and data entry, freeing up healthcare professionals to focus more on patient care. This increased efficiency is reflected in shorter waiting times for patients, as optimized scheduling and resource allocation ensure that healthcare providers can manage their workloads more effectively. The integration of these technologies thus not only improves the operational efficiency of healthcare facilities but also enhances the patient experience by reducing delays and improving the quality of care provided.

#### **4.1.1 Developed Web Application**

The Ethio-Virtual Healthcare web application platform leverages a modern and efficient tech stack to deliver a user-friendly and scalable solution. The React-based frontend utilizes component-based architecture for maintainability and performance, while the Django backend ensures robust data management and API integration. The application adheres to responsive design principles, ensuring a seamless user experience across various devices.

The integration of PostgreSQL as the database management system guarantees secure and efficient data storage and retrieval.

This combination of technologies establishes a solid foundation for the *Ethio-Virtual Healthcare* platform, empowering it to deliver virtual healthcare services effectively.

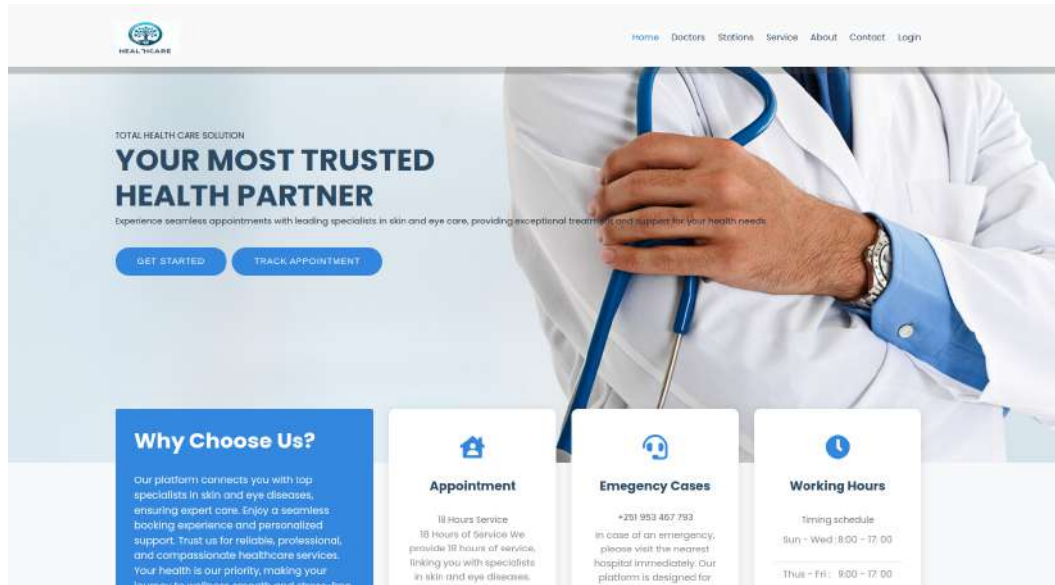


Figure 4.1: Deployed Website

#### 4.1.2 Trained Machine Learning Model

The Ethio-Virtual Healthcare web application leverages machine learning for image-based disease detection. This project emphasizes the importance of high-quality, diverse data for training effective models.

1. Skin Cancer: The HAM10000 dataset provides a rich collection of dermoscopic images for various skin conditions, enabling model development to differentiate between benign and malignant lesions.
2. Eye Disease: The ODIR dataset offers real-world data with color fundus photographs of various eye diseases, allowing the model to learn to identify a broad spectrum of eye conditions.

ResNet-50, a deep learning architecture, serves as the foundation for the machine learning models due to its effectiveness in addressing the vanishing gradient problem and enabling efficient training. The model undergoes meticulous evaluation using metrics like accuracy and confusion matrices to assess its performance and identify areas for improvement.



## **4.2 Discussions**

In the face of limited availability of specialists, particularly in fields such as ophthalmology and dermatology, the integration of AI-enabled platforms offers a promising solution. These platforms can assist local physicians in making accurate decisions and ensure that severe cases receive timely attention from specialists. Discuss the implications of the findings, including any practical applications or theoretical contributions.

The results have been discussed with key findings. The next chapter along with conclusion, further recommendation to implement it in better way is suggested.

# Chapter 5

## CONCLUSION and RECOMMENDATION

### 5.1 Conclusion

In conclusion, the implementation of AI and ML algorithms has markedly improved the diagnostic accuracy for eye and skin diseases, leading to earlier and more precise diagnoses. The adoption of these technologies has resulted in a significant reduction in diagnostic errors, thereby enhancing patient outcomes through timely and appropriate treatments. Additionally, the creation of Virtual Healthcare Centers has expanded access to specialized care, particularly for patients in underserved areas, by facilitating remote consultations. This approach has successfully reduced travel time and costs, making specialized healthcare more accessible and convenient for a broader population. The efficiency of healthcare services has also been enhanced through the automation of routine tasks, allowing healthcare professionals to utilize their time more effectively and reducing waiting times for patients through optimized scheduling and resource allocation.

The project has demonstrated significant advancements in both the accessibility and efficiency of healthcare services. By leveraging AI and ML, the quality of diagnostics has been elevated, and through virtual platforms, specialized human resources have become more accessible to those in need, regardless of geographical barriers. These improvements collectively contribute to a more equitable and efficient healthcare system.

### 5.2 Recommendations

Looking ahead, there are several areas for future development that can further enhance the benefits observed in this project. One key recommendation is the integration of a com-

prehensive hospital management system (HMS). An HMS can streamline various administrative and clinical processes within healthcare facilities, offering a unified platform for managing patient records, appointments, billing, and other critical functions. This integration can lead to better coordination of care, improved data management, and more efficient hospital operations.

Another significant area for future work is the incorporation of an e-pharmacy system with payment integration. This development would enable patients to order medications online and have them delivered to their homes, thereby enhancing convenience and adherence to prescribed treatments. Payment integration would simplify the transaction process, making it easier for patients to access and pay for their medications. Additionally, the e-pharmacy could be linked with the diagnostic systems to provide personalized medication recommendations based on the AI-driven diagnosis, ensuring a seamless and comprehensive healthcare experience.

In conclusion, while the current project has achieved remarkable improvements in diagnostic accuracy and access to specialized care, future work focusing on the integration of hospital management systems and e-pharmacy solutions with payment capabilities holds the potential to further transform and enhance the healthcare landscape. These advancements would not only streamline healthcare delivery but also ensure that patients receive comprehensive, timely, and convenient care, thus continuing to build on the success of the project.

## REFERENCES

- [1] P. Webster. “Virtual Health Care in the Era of COVID-19.” In: *The Lancet* (2020).
- [2] Giulio Perrone et al. “Telemedicine during Covid-19 pandemic: Advantage or critical issue?” In: *Medico-Legal Journal* 88.2 (2020), pp. 76–77.
- [3] Abraha Woldemichael et al. “Availability and inequality in accessibility of health centre-based primary healthcare in Ethiopia”. In: *PloS one* 14.3 (2019), e0213896.
- [4] Yosef Haile et al. “Membership dropout rates and associated factors in a community-based health insurance scheme in southern Ethiopia: a mixed method study”. In: *Frontiers in Health Services* 3 (2023).
- [5] Agmasie Damtew Walle et al. “Exploring facilitators and barriers of the sustainable acceptance of E-health system solutions in Ethiopia: a systematic review”. In: *PLoS One* 18.8 (2023), e0287991.
- [6] Cui Chen and Mao Liu. “Achievements and challenges of the healthcare system in China”. In: *Cureus* 15.5 (2023).
- [7] Marco Dettori and Paolo Castiglia. *COVID-19 and digital health: evolution, perspectives and opportunities*. 2022.
- [8] Masresha Derese Tegegne and Sisay Maru Wubante. “Identifying barriers to the adoption of information communication technology in ethiopian healthcare systems. a systematic review”. In: *Advances in Medical Education and Practice* 13 (2022), p. 821.
- [9] Yifru Berhan. “Medical doctors profile in Ethiopia: production, attrition and retention. In memory of 100-years Ethiopian modern medicine & the new Ethiopian millennium.” In: *Ethiopian medical journal* 46 (2008), pp. 1–77.
- [10] WHO. *Indicators Details*. 2022. URL: [https://www.who.int/data/gho/data/indicators/indicator-details/GHO/medical-doctors-\(per-10-000-population\)](https://www.who.int/data/gho/data/indicators/indicator-details/GHO/medical-doctors-(per-10-000-population)) (visited on 2024).
- [11] Kelvin J Bwalya. “Next wave of tele-medicine: virtual presence of medical personnel”. In: *Health Information Systems and the Advancement of Medical Practice in Developing Countries*. IGI Global, 2017, pp. 168–180.
- [12] Sandra Dohmen, Alina Wahl, and Carina Benstoem. “Digital Based Intensive Care Networks”. In: *Anesthesiologie, Intensivmedizin, Notfallmedizin, Schmerztherapie: AINS* 57.3 (2022), pp. 211–220.
- [13] Ltd. Shanggong Medical Technology Co. *Ocular Disease Recognition*. 2020. URL: <https://www.kaggle.com/datasets/andrewmvd/ocular-disease-recognition-odir5k> (visited on 2024).

- [14] Noel Codella et al. *Skin Cancer MNIST: HAM10000*. 2018. URL: <https://www.kaggle.com/datasets/kmader/skin-cancer-mnist-ham10000/data> (visited on 2024).
- [15] *Ethio-Virtual Healthcare*. 2024. URL: <https://ethio-virtual-healthcare-center.netlify.app/> (visited on 2024).
- [16] Frederick Ng, Ruihan Jiang, and James Chow. “Predicting radiation treatment planning evaluation parameter using artificial intelligence and machine learning”. In: *IOP SciNotes* 1 (July 2020), p. 014003. DOI: 10.1088/2633-1357/ab805d.
- [17] *ResNet 50 Architechture*. 2024. URL: <https://media.geeksforgeeks.org/wp-content/uploads/20200424011138/ResNet.PNG> (visited on 2024).

## APPENDIX A

### Code for Machine Learning Model

```
import tensorflow as tf
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
import os
import cv2
import random
from tqdm import tqdm
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing.image import ImageDataGenerator

data = pd.read_csv("full_df.csv")
data.head()
```

Figure 1: Importing Libraries and Reading Data

```
# creating the model
from tensorflow.keras.applications import ResNet50
import tensorflow as tf
resnet = ResNet50(weights="imagenet", include_top=False, input_shape=(image_size, image_size, 3))

for layer in resnet.layers:
    layer.trainable = False

from tensorflow.keras import Sequential
from tensorflow.keras.layers import Flatten, Dense
model = Sequential()
model.add(resnet)
model.add(Flatten())
model.add(Dense(256, activation="relu"))
model.add(tf.keras.layers.BatchNormalization())
model.add(Dense(8, activation="softmax"))

model.summary()
```

Figure 2: Creating ResNet50 Model for Eye Diagnosis

```
# after creating the model we will train it.

model.compile(optimizer="adam",loss="categorical_crossentropy",metrics=["accuracy"])
#u might have to apply train_generator and test_generator
history = model.fit(x_train,y_train_cat,batch_size=32,epochs=1)
print(history)
```

Figure 3: Training the Model for Eye Diagnosis

```
# after train the model we can see it's evaluation
loss,accuracy = model.evaluate(x_test,y_test_cat)
print("Accuracy:",accuracy)

from sklearn.metrics import confusion_matrix,classification_report,accuracy_score
# y_pred = np.array((model.predict(x_test) > 0.5).astype("int32"))

y_pred = []
for i in model.predict(x_test):
    y_pred.append(np.argmax(np.array(i)).astype("int32"))

print(y_pred)
print(accuracy_score(y_test,y_pred))
```

Figure 4: Model Evaluation

```
[ ] import tensorflow as tf
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
import os
import cv2
import seaborn as sns
import random
from glob import glob
from tqdm import tqdm
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

Figure 5: Creating ResNet50 Model for Eye Diagnosis

```
🔍 skin_df = pd.read_csv('HAM10000_metadata.csv')

# Creating New Columns for better readability

skin_df['path'] = skin_df['image_id'].map(imageid_path_dict.get)
skin_df['cell_type'] = skin_df['dx'].map(lesion_type_dict.get)
skin_df['cell_type_idx'] = pd.Categorical(skin_df['cell_type']).codes
skin_df
```

Figure 6: Training the Model for Eye Diagnosis

```

sns.set_style('whitegrid')
colors = ['#87ace8', '#e3784d', 'green']
fig, axes = plt.subplots(figsize=(12, 8))

ax = sns.countplot(x='sex', data=skin_df)
for container in ax.containers:
    ax.bar_label(container)
plt.title('Gender-wise Distribution')
plt.xticks(rotation=45)
plt.show()

```

Figure 7: Model Evaluation

```

loss, accuracy = model.evaluate(x_test, y_test_cat)
print("Accuracy:", accuracy)

from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
# y_pred = np.array((model.predict(x_test) > 0.5).astype("int32"))

y_pred = []
for i in model.predict(x_test):
    y_pred.append(np.argmax(np.array(i)).astype("int32"))

print(y_pred)
print(accuracy_score(y_test, y_pred))

```

Figure 8: Training the Model for Eye Diagnosis

```

sns.set_style('whitegrid')
fig, axes = plt.subplots(figsize=(12, 8))
ax = sns.countplot(x='localization', data=skin_df, hue='localization', order = skin_df['localization'].value_counts().index, palette=colors)
for container in ax.containers:
    ax.bar_label(container)
plt.title('Localization Area Frequencies')
plt.xticks(rotation=45)
plt.show()

```

Figure 9: Cell Type Frequency



## APPENDIX B

### Output for Machine Learning

| Layer (type)                              | Output Shape       | Param #  |
|-------------------------------------------|--------------------|----------|
| resnet50 (Functional)                     | (None, 7, 7, 2048) | 23587712 |
| flatten (Flatten)                         | (None, 100352)     | 0        |
| dense (Dense)                             | (None, 256)        | 25690368 |
| batch_normalization (Batch Normalization) | (None, 256)        | 1024     |
| dense_1 (Dense)                           | (None, 8)          | 2056     |
| Total params: 49281160 (187.99 MB)        |                    |          |
| Trainable params: 25692936 (98.01 MB)     |                    |          |
| Non-trainable params: 23588224 (89.98 MB) |                    |          |

Figure 10: Skin Disease Model Summary Output

```
135/135 [=====] - 1103s 8s/step - loss: 1.3411 - accuracy: 0.5582  
<keras.src.callbacks.History object at 0x789504670370>
```

Figure 11: Skin Disease Model Test on Training Data

```

34/34 [=====] - 245s 7s/step - loss: 0.9891 - accuracy: 0.6630
Accuracy: 0.6629629731178284
34/34 [=====] - 233s 7s/step
[3, 6, 2, 2, 5, 2, 2, 4, 5, 2, 2, 5, 2, 1, 6, 2, 6, 2, 2, 4, 5, 1, 3, 2, 2, 5, 2, 5, 2, 2, 2, 1, 1,
0.662962962962963

```

Figure 12: Skin Disease Model Test on Test Data

| Layer (type)                                | Output Shape       | Param #  |
|---------------------------------------------|--------------------|----------|
| resnet50 (Functional)                       | (None, 7, 7, 2048) | 23587712 |
| flatten (Flatten)                           | (None, 100352)     | 0        |
| dense (Dense)                               | (None, 256)        | 25690368 |
| batch_normalization (Batch Normalization)   | (None, 256)        | 1024     |
| dropout (Dropout)                           | (None, 256)        | 0        |
| dense_1 (Dense)                             | (None, 128)        | 32896    |
| batch_normalization_1 (Batch Normalization) | (None, 128)        | 512      |
| dense_2 (Dense)                             | (None, 7)          | 903      |
| =====                                       |                    |          |
| Total params: 49313415 (188.12 MB)          |                    |          |
| Trainable params: 25724935 (98.13 MB)       |                    |          |
| Non-trainable params: 23588480 (89.98 MB)   |                    |          |

Figure 13: Eye Disease Model Summary Output

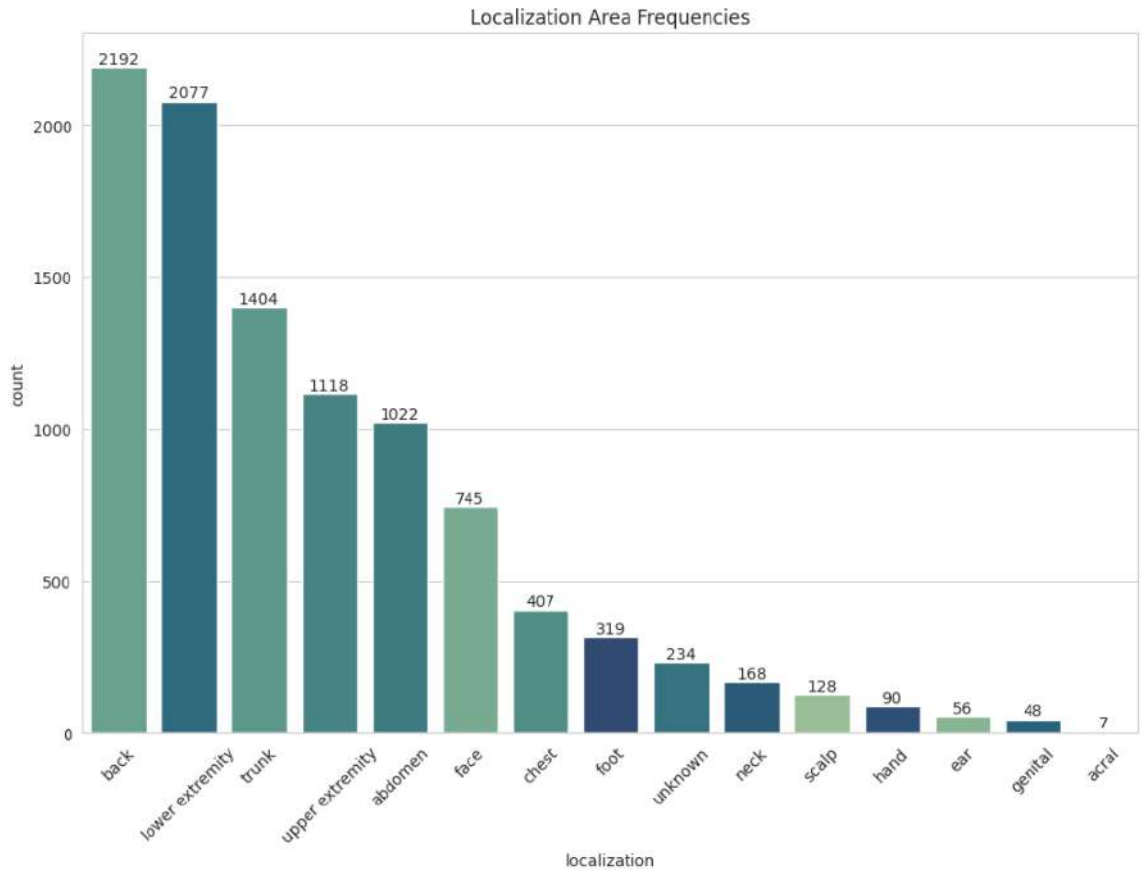


Figure 14: Localization Area Frequencies

|       | lesion_id   | image_id     | dx    | dx_type | age  | sex    | localization | path                                    | cell_type                     | cell_type_idx |
|-------|-------------|--------------|-------|---------|------|--------|--------------|-----------------------------------------|-------------------------------|---------------|
| 0     | HAM_0000118 | ISIC_0027419 | bkl   | histo   | 80.0 | male   | scalp        | ham10000_images_part_1/ISIC_0027419.jpg | Benign keratosis-like lesions | 2             |
| 1     | HAM_0000118 | ISIC_0025030 | bkl   | histo   | 80.0 | male   | scalp        | ham10000_images_part_1/ISIC_0025030.jpg | Benign keratosis-like lesions | 2             |
| 2     | HAM_0002730 | ISIC_0026769 | bkl   | histo   | 80.0 | male   | scalp        | ham10000_images_part_1/ISIC_0026769.jpg | Benign keratosis-like lesions | 2             |
| 3     | HAM_0002730 | ISIC_0025661 | bkl   | histo   | 80.0 | male   | scalp        | ham10000_images_part_1/ISIC_0025661.jpg | Benign keratosis-like lesions | 2             |
| 4     | HAM_0001466 | ISIC_0031633 | bkl   | histo   | 75.0 | male   | ear          | ham10000_images_part_2/ISIC_0031633.jpg | Benign keratosis-like lesions | 2             |
| ...   | ...         | ...          | ...   | ...     | ...  | ...    | ...          | ...                                     | ...                           | ...           |
| 10010 | HAM_0002867 | ISIC_0033084 | akiec | histo   | 40.0 | male   | abdomen      | ham10000_images_part_2/ISIC_0033084.jpg | Actinic keratoses             | 0             |
| 10011 | HAM_0002867 | ISIC_0033550 | akiec | histo   | 40.0 | male   | abdomen      | ham10000_images_part_2/ISIC_0033550.jpg | Actinic keratoses             | 0             |
| 10012 | HAM_0002867 | ISIC_0033536 | akiec | histo   | 40.0 | male   | abdomen      | ham10000_images_part_2/ISIC_0033536.jpg | Actinic keratoses             | 0             |
| 10013 | HAM_0000239 | ISIC_0032854 | akiec | histo   | 80.0 | male   | face         | ham10000_images_part_2/ISIC_0032854.jpg | Actinic keratoses             | 0             |
| 10014 | HAM_0003521 | ISIC_0032258 | mel   | histo   | 70.0 | female | back         | ham10000_images_part_2/ISIC_0032258.jpg | Melanoma                      | 5             |

10015 rows × 10 columns

Figure 15: The Skin Data Set Partially



Figure 16: Eye Output

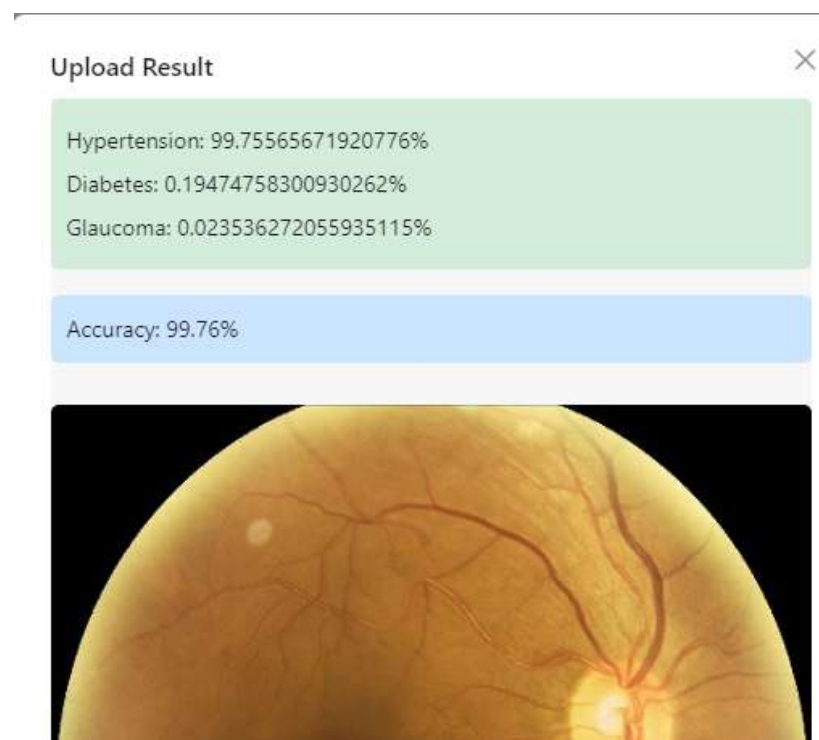


Figure 17: Eye Output